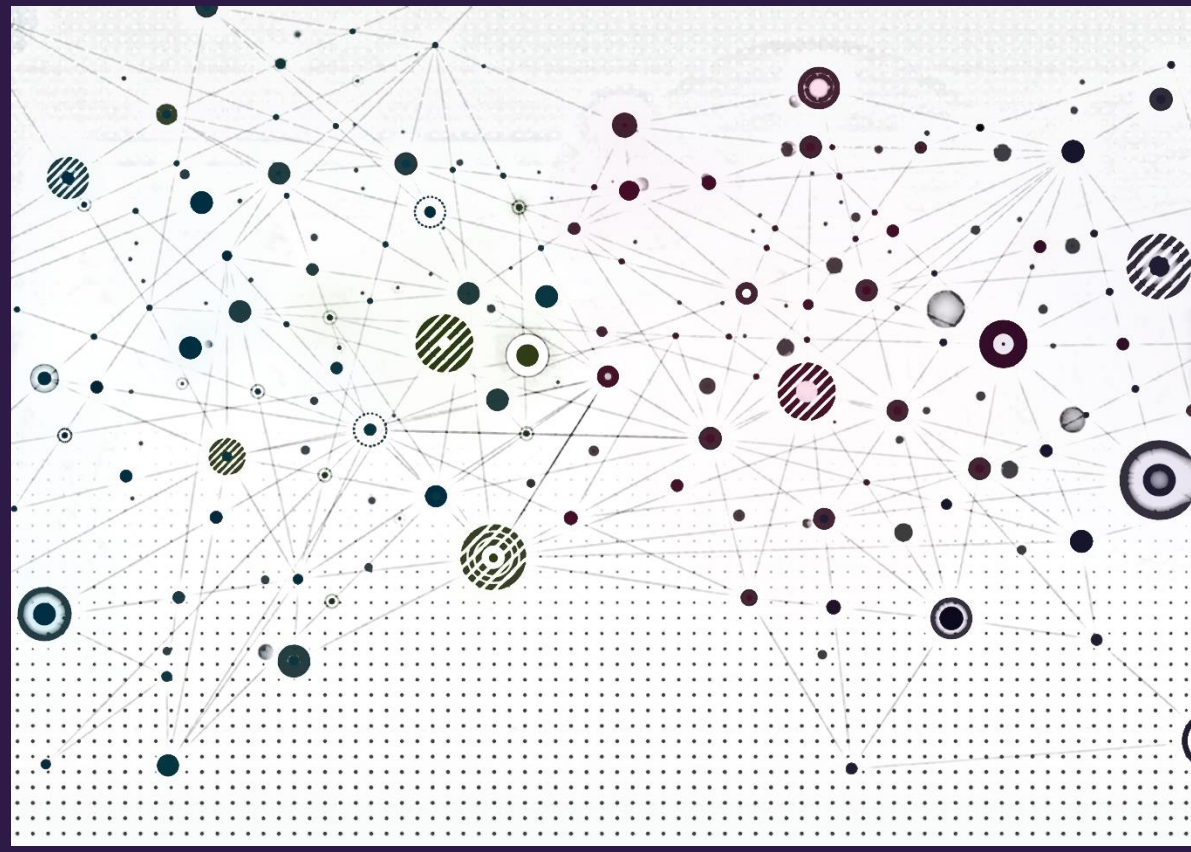
KG foundation models



Overview

- What is a foundation model?
- Knowledge graph (KG) review
- Standard KG completion models fail as KG foundation models
- Relational GNNs mostly fail as KG foundation models
- InGram – a conditional KG foundation model

School of COMPUTING Clemson University

What is a foundation model?

What is a foundation model?

If you ask your favorite LLM, you'll get something like:

A **foundation model** is a large-scale machine learning model trained on broad, diverse data that can be adapted to a wide range of downstream tasks, rather than being designed for a single specific purpose.

Key characteristics include:

Scale: These models are typically trained on massive datasets using substantial computational resources, resulting in billions (or even trillions) of parameters.

Transfer learning capability: Rather than training from scratch for each application, foundation models can be fine-tuned, prompted, or adapted with relatively little additional data or compute for specific tasks.

Broad knowledge: They learn general patterns, representations, and capabilities from diverse training data that prove useful across multiple domains and tasks.

Emergence: At sufficient scale, these models often exhibit emergent capabilities—abilities that weren't explicitly programmed but arise from the training process.

What is a foundation model?

If you ask your favorite LLM, you'll get something like:

A **foundation model** is a large-scale machine learning model trained on broad, diverse data that can be adapted to a wide range of downstream tasks, rather than being designed for a single specific purpose.

Key characteristics include:

Scale: These models are typically trained on massive datasets using substantial computational resources, resulting in billions (or even trillions) of parameters.

Transfer learning capability: Rather than training from scratch for each application, foundation models can be fine-tuned, prompted, or adapted with relatively little additional data or compute for specific tasks.

Broad knowledge: They learn general patterns, representations, and capabilities from diverse training data that prove useful across multiple domains and tasks.

Emergence: At sufficient scale, these models often exhibit emergent capabilities—abilities that weren't explicitly programmed but arise from the training process.

Subjective terms describing implementation details that are likely to change over time (remember *big data?*)

Isn't this just transfer learning?

What is a foundation model?

An alternative definition:

A **foundation model** is a machine learning model that exhibits sufficient task diversity and generalization that it can perform novel tasks with different formal structures from its training objective, making zero-shot or few-shot application the reasonable first approach for novel problems.

What is a foundation model?

An alternative definition:

A **foundation model** is a machine learning model that exhibits sufficient task diversity and generalization that it can perform novel tasks with different formal structures from its training objective, making zero-shot or few-shot application the reasonable first approach for novel problems.

- Task diversity – performing a task with the same formal definition in different domains (e.g., sentiment analysis on movies vs. product reviews)

What is a foundation model?

An alternative definition:

A **foundation model** is a machine learning model that exhibits sufficient task diversity and generalization that it can perform novel tasks with different formal structures from its training objective, making zero-shot or few-shot application the reasonable first approach for novel problems.

- Task diversity – performing a task with the same formal definition in different domains (e.g., sentiment analysis on movies vs. product reviews)
- Task generalization – performing a task with a different formal definition which typically includes different evaluation criteria (e.g., next token prediction vs. question answering)

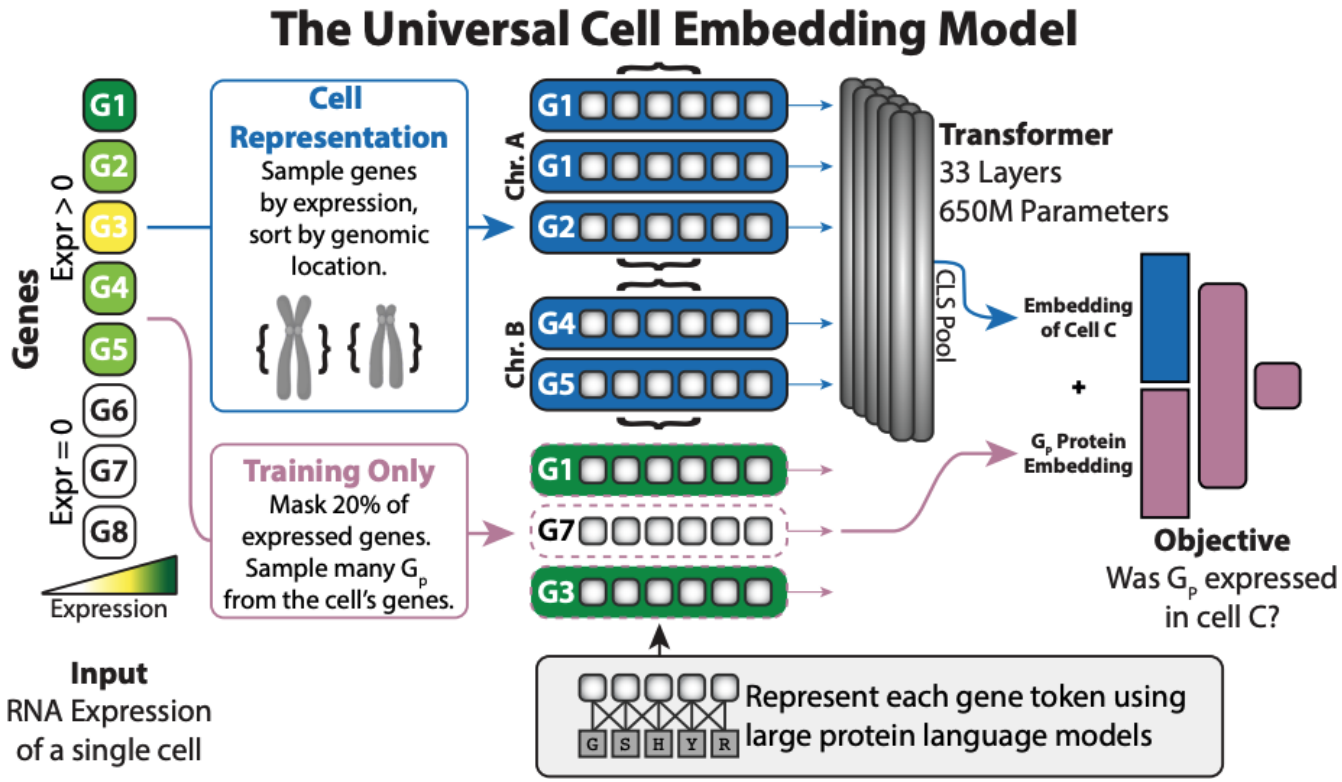
Large language models (LLMs)

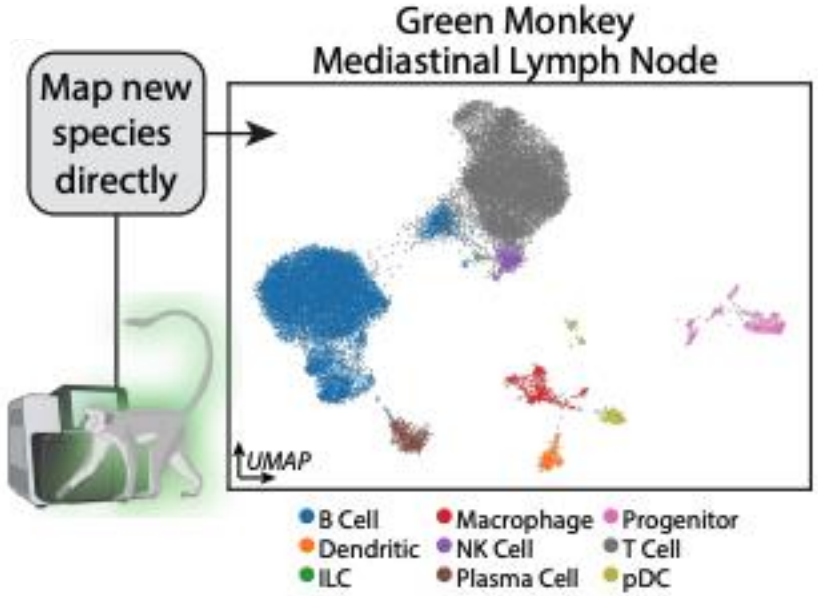
Figure 1.2: Larger models make increasingly efficient use of in-context information. We show in-context learning performance on a simple task requiring the model to remove random symbols from a word, both with and without a natural language task description (see Sec. 3.9.2). The steeper "in-context learning curves" for large models demonstrate improved ability to learn a task from contextual information. We see qualitatively similar behavior across a wide range of tasks.

Computer vision models

Figure 1. A building foundation — **Image: DALLE 2**

Cell biology models





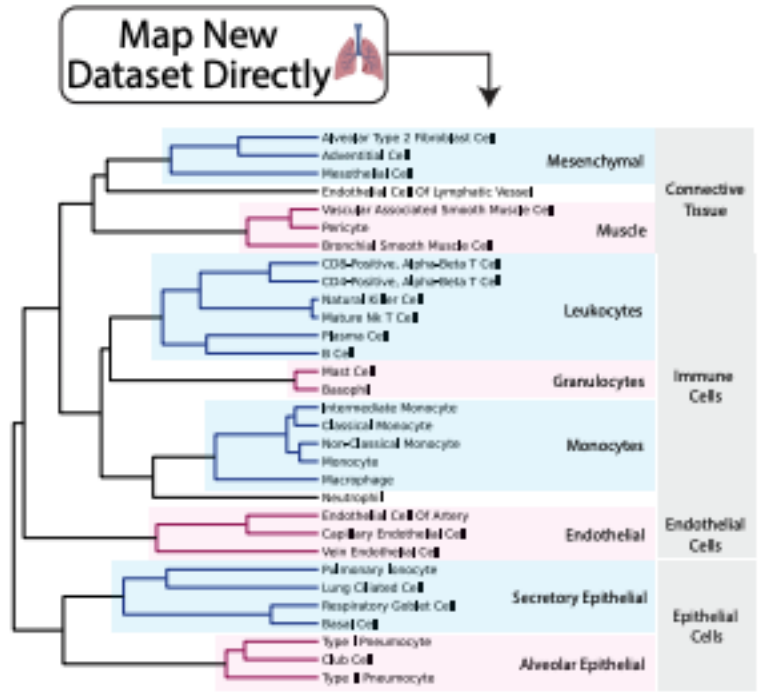


Image credit: Rosen et al., 2023, bioRxiv

Today's foundation models

- Pretrained using self-supervision with *large* amounts of data
- Capable of zero-shot or few-shot learning on tasks that differ from pre-training
- Generalize to entities (domains) not observed during training

Graph foundation models?

- Many types of graphs with different structures, semantics, and feature data
- Many tasks in different domains

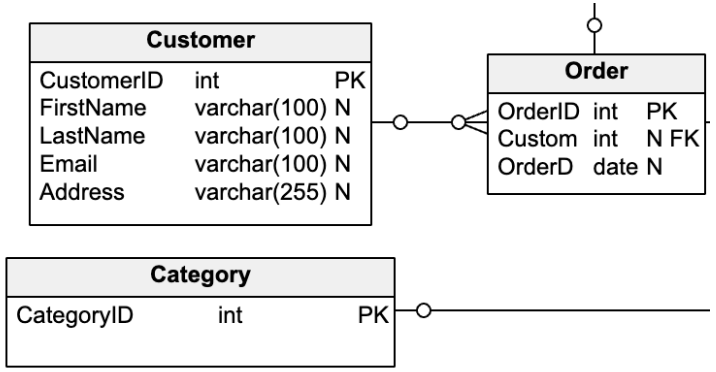


Image credit: vertabelo

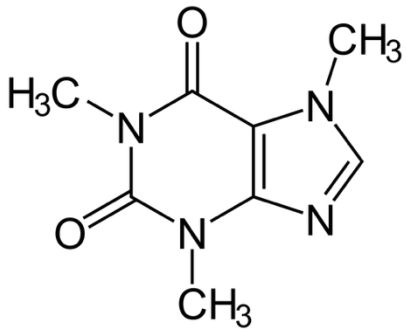


Image credit: Chemistry StackExchange

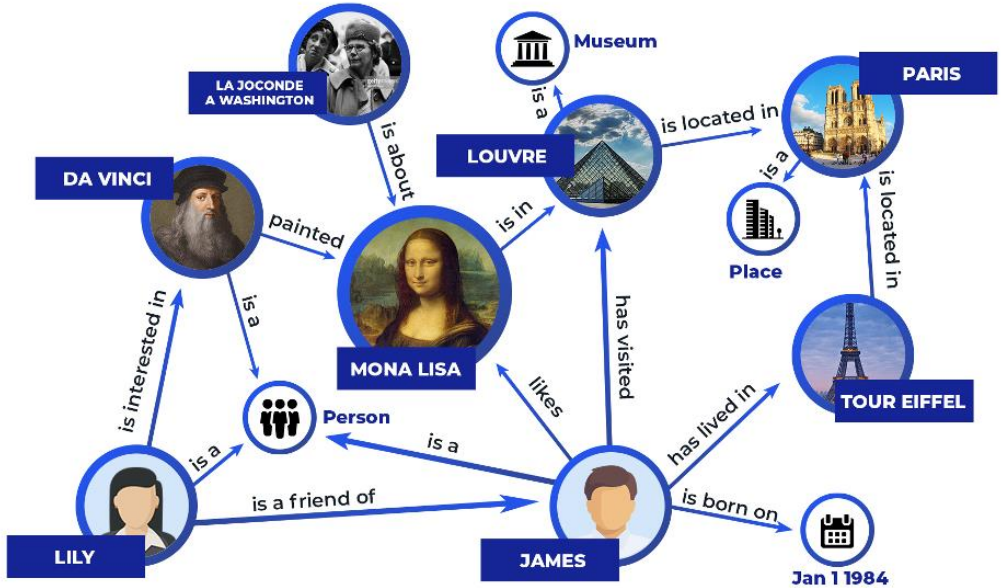
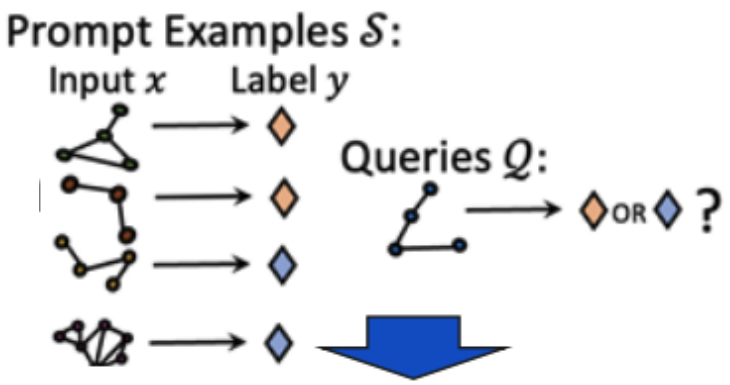


Image credit: Zilliz

PRODIGY – a step toward graph foundation models

- Unified framework for few-shot classification learning on graphs
- Agnostic to graph type
- Constraints
  - Requires data graph nodes to have input features ⟶ node features must be represented in the same feature space (e.g., text embedding)
  - Requires retraining in different domains where node features are not represented with text or text is out of sample from training data



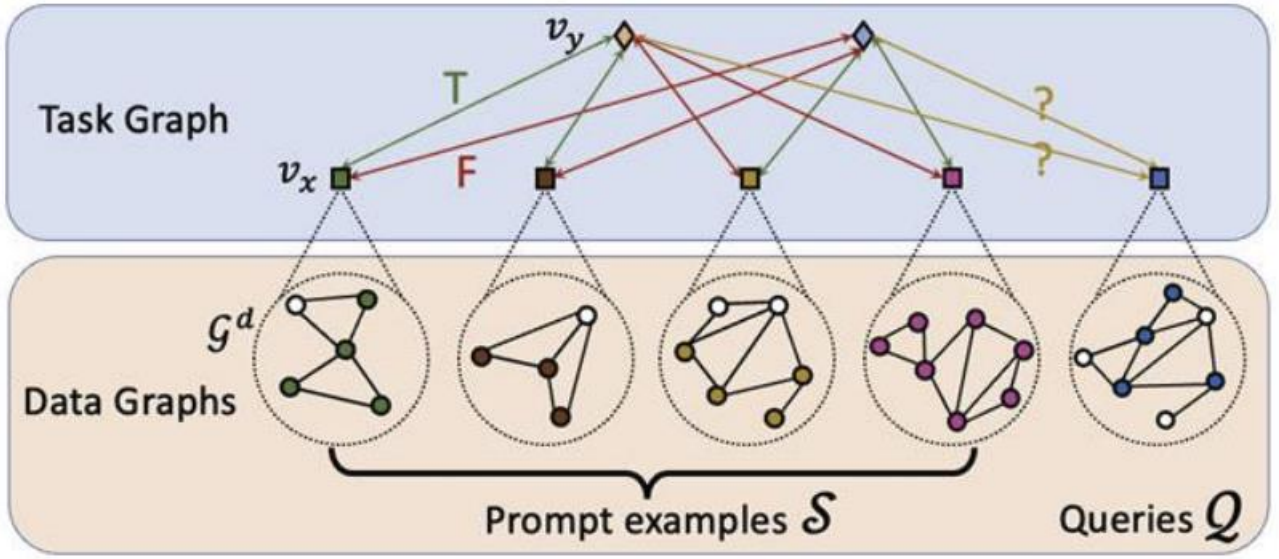


Image credit: Huang et all, 2023, NeurIPS

School of COMPUTING Clemson University

Knowledge graph review

Knowledge graphs

- Heterogeneous graphs that represent entities, types, and relationships
- Nodes are *entities*
  - Nodes are labeled with their types
  - Nodes do **not** have features
- Edges between nodes indicate specific relationships between entities, often directional
- Typically represented by triples (head, relation, tail) where head (h) has relation (r) with tail (t)

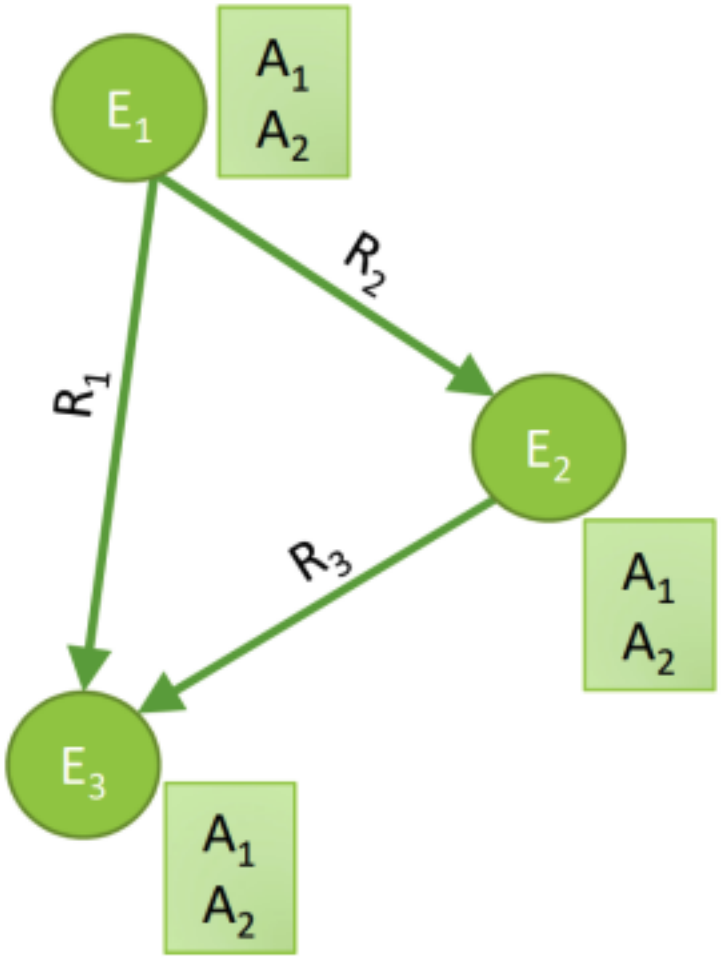
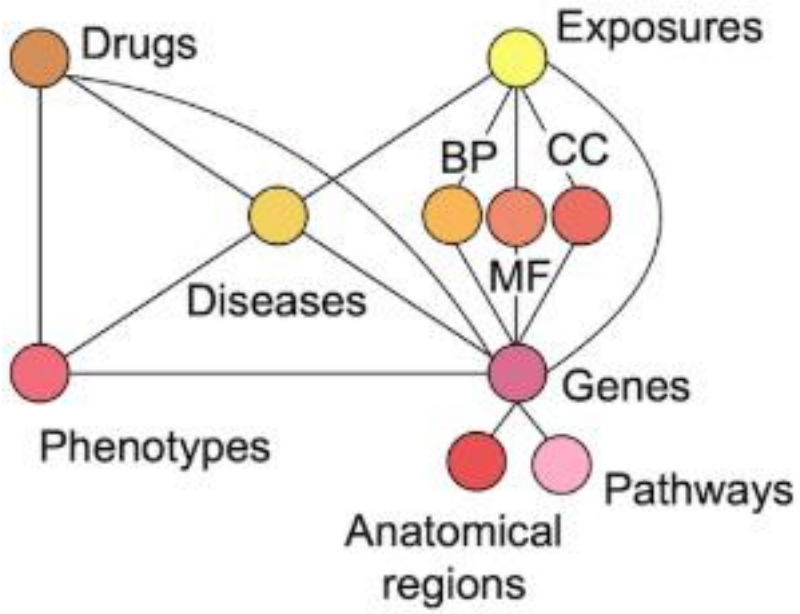
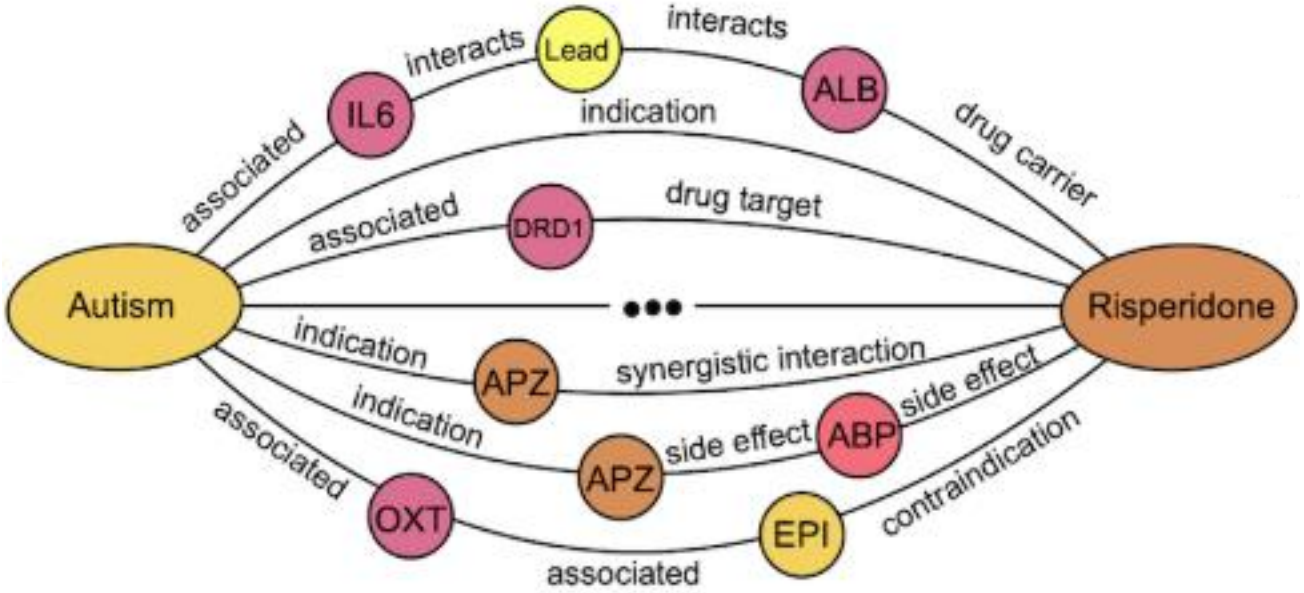


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Biomedical knowledge graphs

- Node types: drugs, biological pathways, diseases, …
- Relation types: associated, interacts, …





Bibliographic networks

- Node types: paper, title, author, conference, year
- Relation types: pubWhere, pubYear, hasTitle, hasAuthor, cite

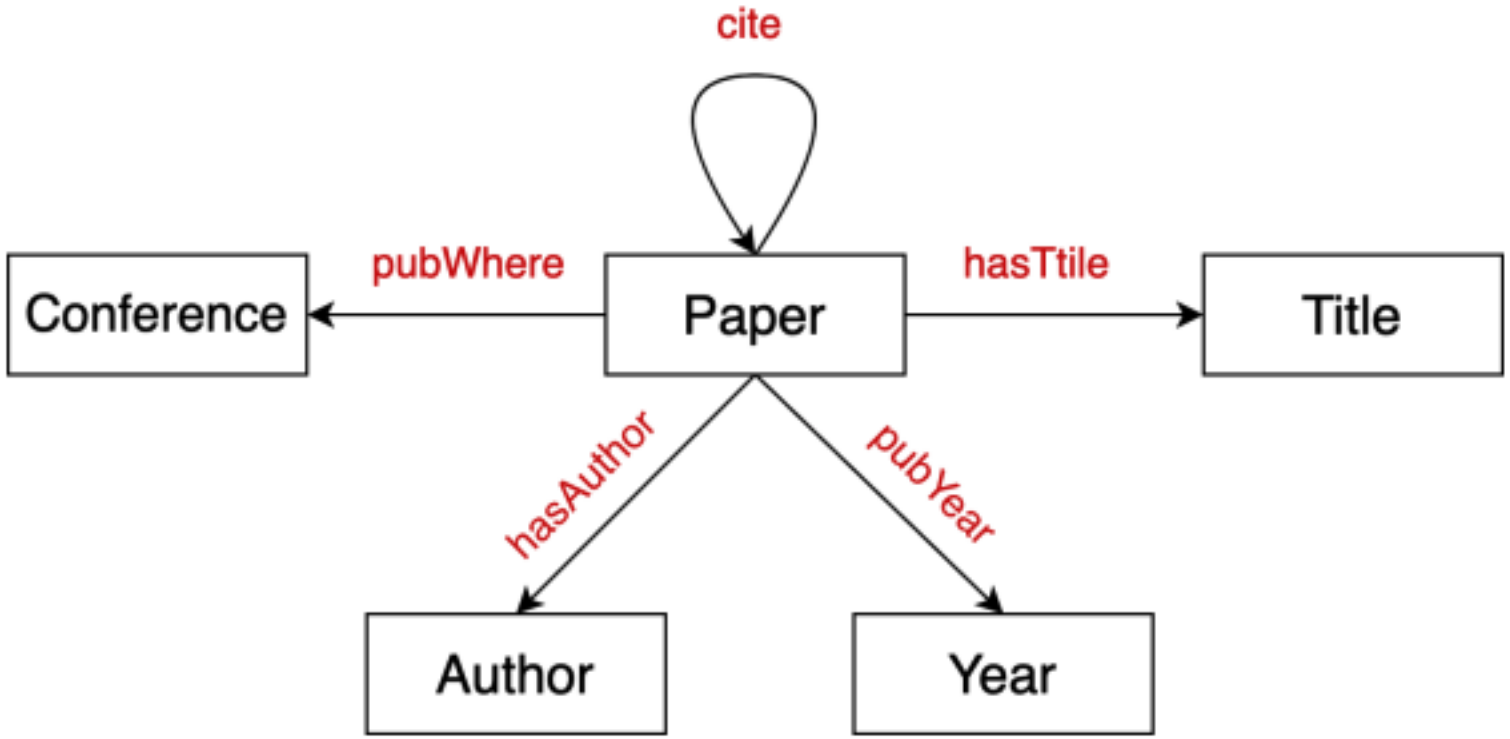


Image credit: Jure Leskovec, http://cs224w.stanford.edu

E-commerce graph

- Node types: user, item, query, location
- Relation types: purchase, visit, guide, search

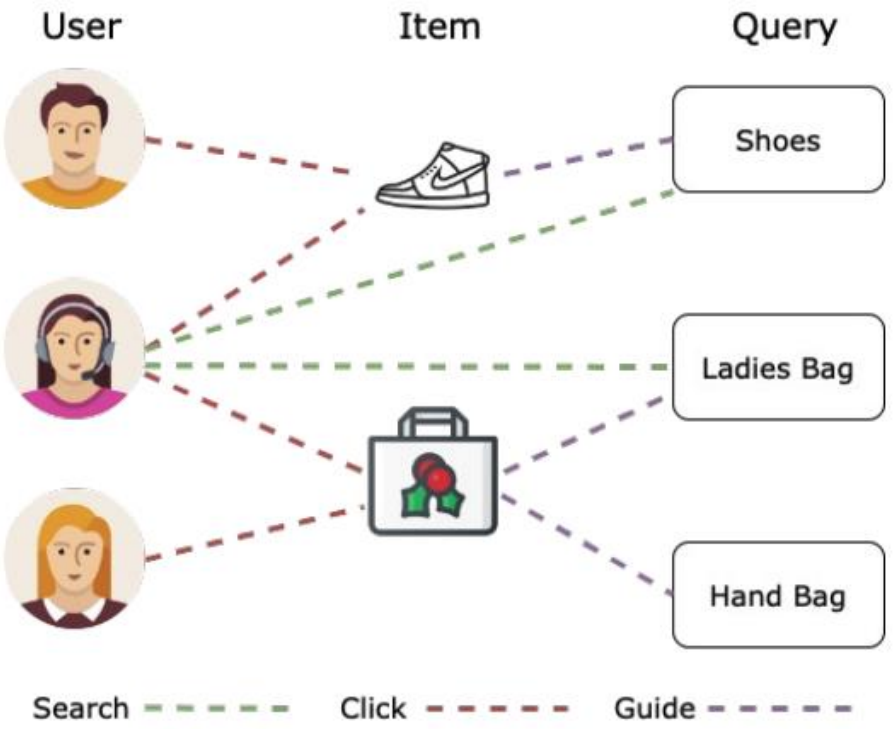


Image credit: Jure Leskovec, http://cs224w.stanford.edu

KG vs natural language foundation models

E-commerce graph example

- Natural language:
  - User searches for shoes. User buys hand-bag.
  - All entities are of the same type (words / tokens)
- Knowledge graph:
  - User, Shoes, Hand-bag ⟶ entities
  - Searches, Buys ⟶ relations
  - Two modalities, heterogeneous constructs requiring different “interpretation”

KG vs natural language foundation models

E-commerce graph example

- Natural language:
  - User searches for shoes. User buys hand-bag.
  - Tokens are connected sequentially
- Knowledge graph:
  - Entities are connected in an arbitrary graph structure that is informative

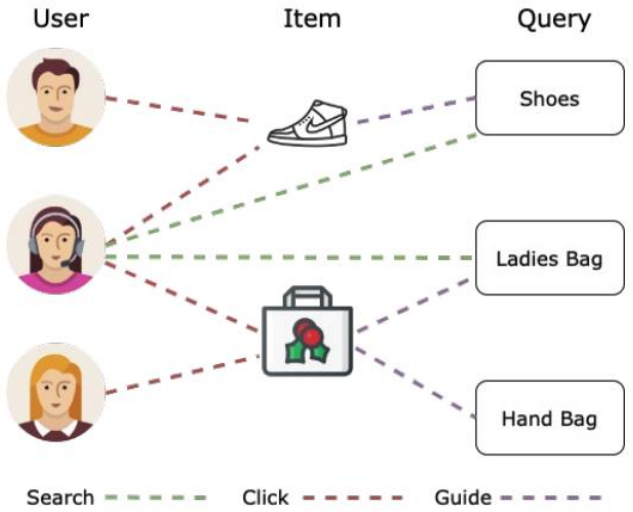


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Toward foundation models for knowledge graphs

- KG foundation model should be able to form embeddings for new entities and relations in any KG
  - Double equivariant architecture – node and edge relabeling only permutes their respective embedding order

Toward foundation models for knowledge graphs

- KG foundation model should be able to form embeddings for new entities and relations in any KG
  - Double equivariant architecture – node and edge relabeling only permutes their respective embedding order
    - Without edge equivariance, model learns “The relation called *employs* connects *organization* to *employee,* and the relation *co-worker* connects *employees”.*

Toward foundation models for knowledge graphs

- KG foundation model should be able to form embeddings for new entities and relations in any KG
  - Double equivariant architecture – node and edge relabeling only permutes their respective embedding order
    - Without edge equivariance, model learns "The relation called *employs* connects *organization* to *employee,* and the relation *co-worker* connects *employees".*
    - With edge equivariance, model learns relations that connect high-degree nodes to medium-degree nodes which are also connected via a symmetric relation are antisymmetric

Toward foundation models for knowledge graphs

- KG foundation model should be able to form embeddings for new entities and relations in any KG
  - Double equivariant architecture – node and edge relabeling only permutes their respective embedding order
    - Without edge equivariance, model learns "The relation called *employs* connects *organization* to *employee,* and the relation *co-worker* connects *employees".*
    - With edge equivariance, model learns relations that connect high-degree nodes to medium-degree nodes which are also connected via a symmetric relation are antisymmetric
- How? One approach
  - Process the graph structure between entities using GNNs
  - Use KG completion as a self-supervised learning task

School of COMPUTING Clemson University

Standard KG completion models fail

Knowledge graph (KG) completion task

- Given a large KG, can we ”complete” it (i.e., add missing edges)
- Task is typically formulated as predicting tails (or consequents)
  - Given (head, relation) predict missing tails

Example task: predict the tail “Science Fiction” for (“J.K. Rowling”, “genre”)

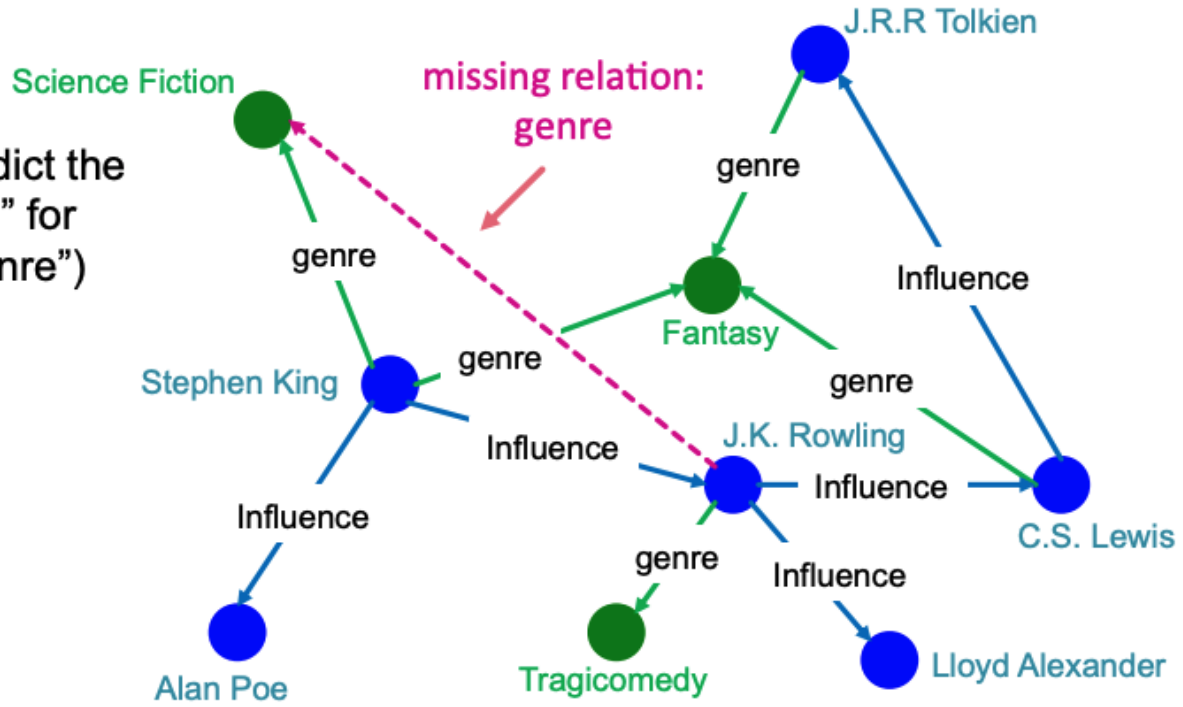
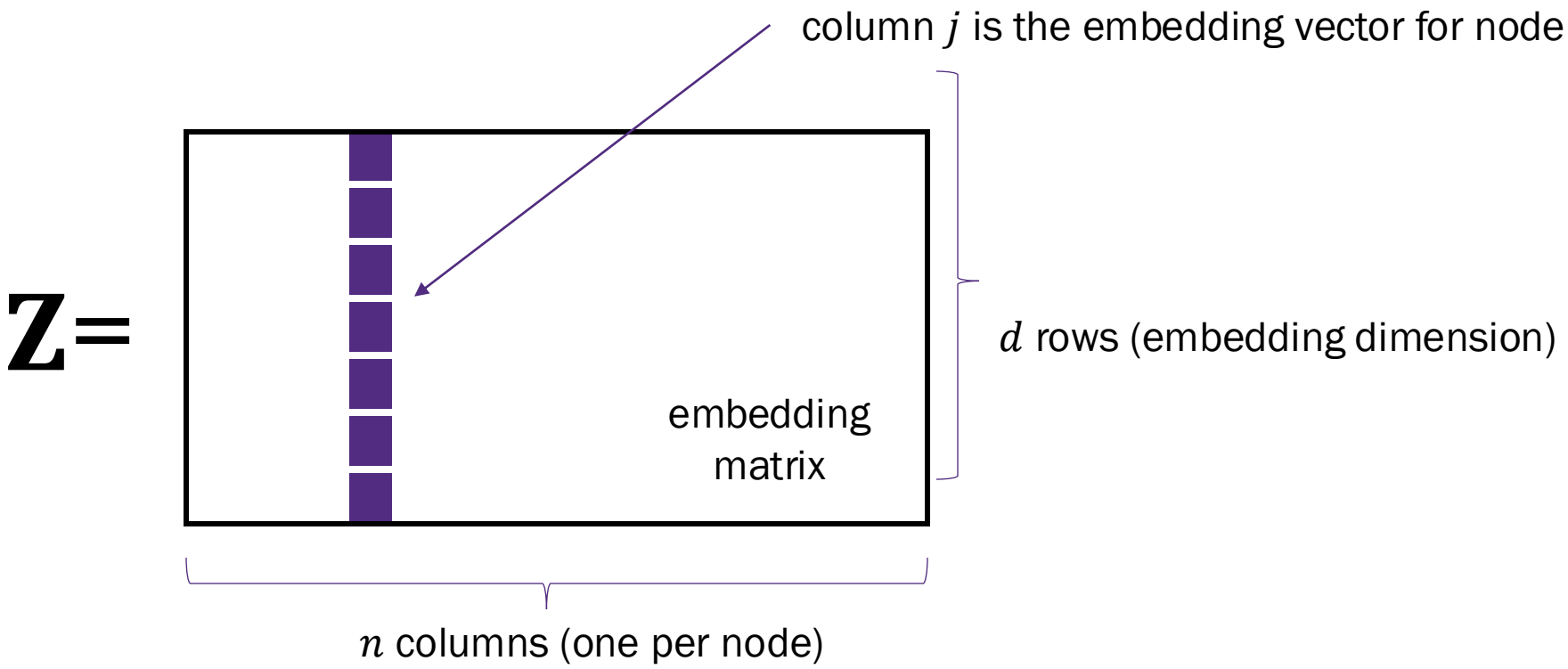


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Transductive methods for KG completion

- "Standard" KG completion methods use shallow embedding methods for computational efficiency on large KGs



KG Completion Models

Model	Score	Embedding	Sym.	Antisym	Inv	Compos	1-to-N
TransE	$-\|\|\mathbf{h}+\mathbf{r}-\mathbf{t}\|\|$	$\mathbf{h},\mathbf{r},\mathbf{t}\in\mathbb{R}^k$	✗	✓	✓	✓	✗
TransR	$-\|\|\mathbf{h}_\perp+r-\mathbf{t}_\perp\|\|$	$\mathbf{h},\mathbf{t}\in\mathbb{R}^d$ $\mathbf{r}\in\mathbb{R}^k$ $\mathbf{M}_r\in\mathbb{R}^{k\times d}$	✓	✓	✓	✓	✓

Standard KG completion models fail as foundation models

- Structure agnostic shallow encoders learn embeddings for entities and relations
  - The entity and relation embeddings are the learnable parameters
- The structure of the KG is used in the loss function
- The KG embeddings are only formed for entities in the single KG
  - There is no way to transfer knowledge between KGs

Standard KG completion models fail as foundation models

- Structure agnostic shallow encoders learn embeddings for entities and relations
  - The entity and relation embeddings are the learnable parameters
- The structure of the KG is used in the loss function
- The KG embeddings are only formed for entities in the single KG
  - There is no way to transfer knowledge between KGs

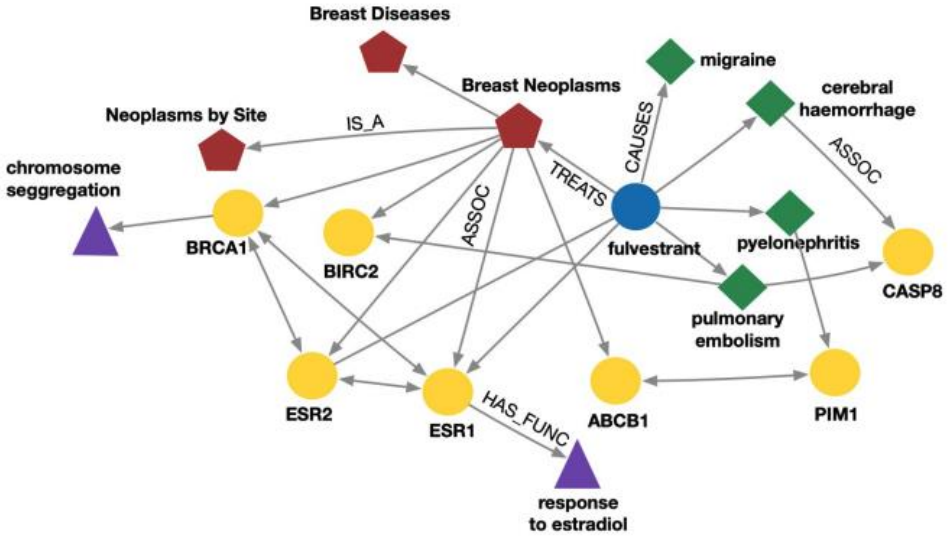


Image credit: Jure Leskovec, http://cs224w.stanford.edu

What happens if a new entity is added to the KG?
Need to retrain the shallow encoder.

School of COMPUTING Clemson University

RGNN models mostly fail

KGs with the same relation types

- What if two KGs have the same relation types? (e.g., E-commerce graphs for two different websites)
- Can we enable transfer learning between them?

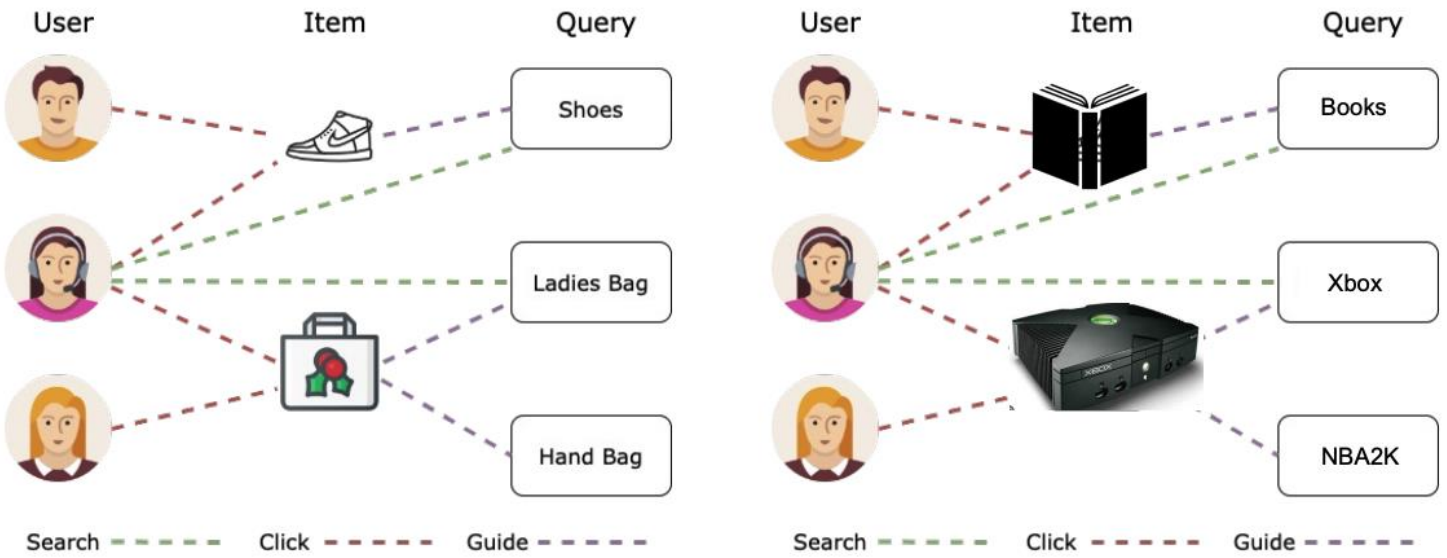


Image credit: Jure Leskovec, http://cs224w.stanford.edu

KGs with the same relation types

- What if two KGs have the same relation types? (e.g., E-commerce graphs for two different websites)
- Can we enable transfer learning between them?
- Observe, a KG is a heterogeneous graph (without node features)
- Use a heterogeneous GNN (e.g., RGCN) with randomly initialized feature vectors

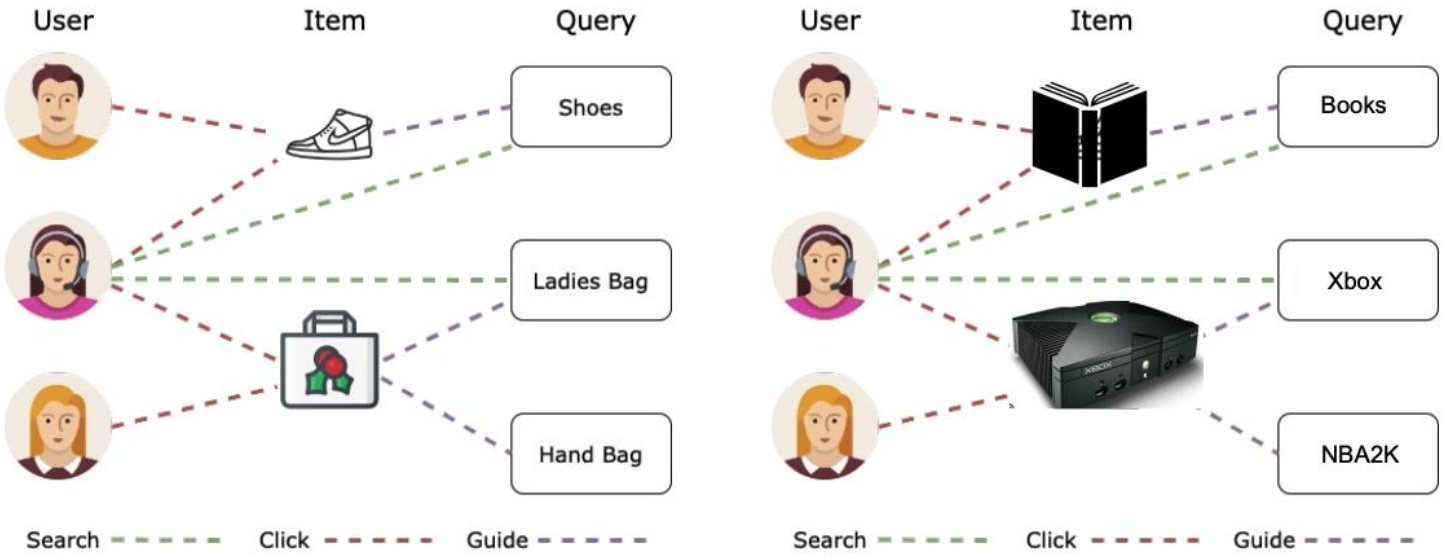
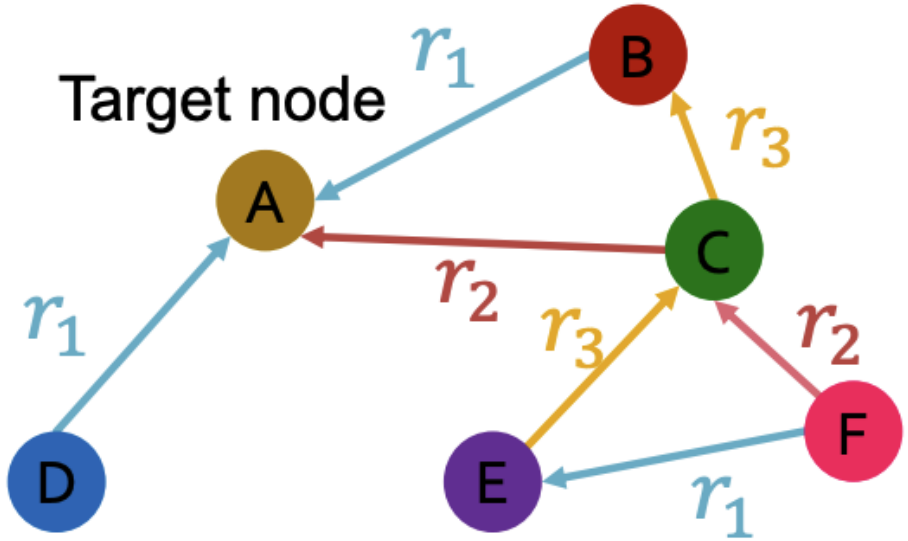


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational graph convolutional networks (RGCN)

- What if the graph has multiple relation types?
- Use different neural networks for different relation types



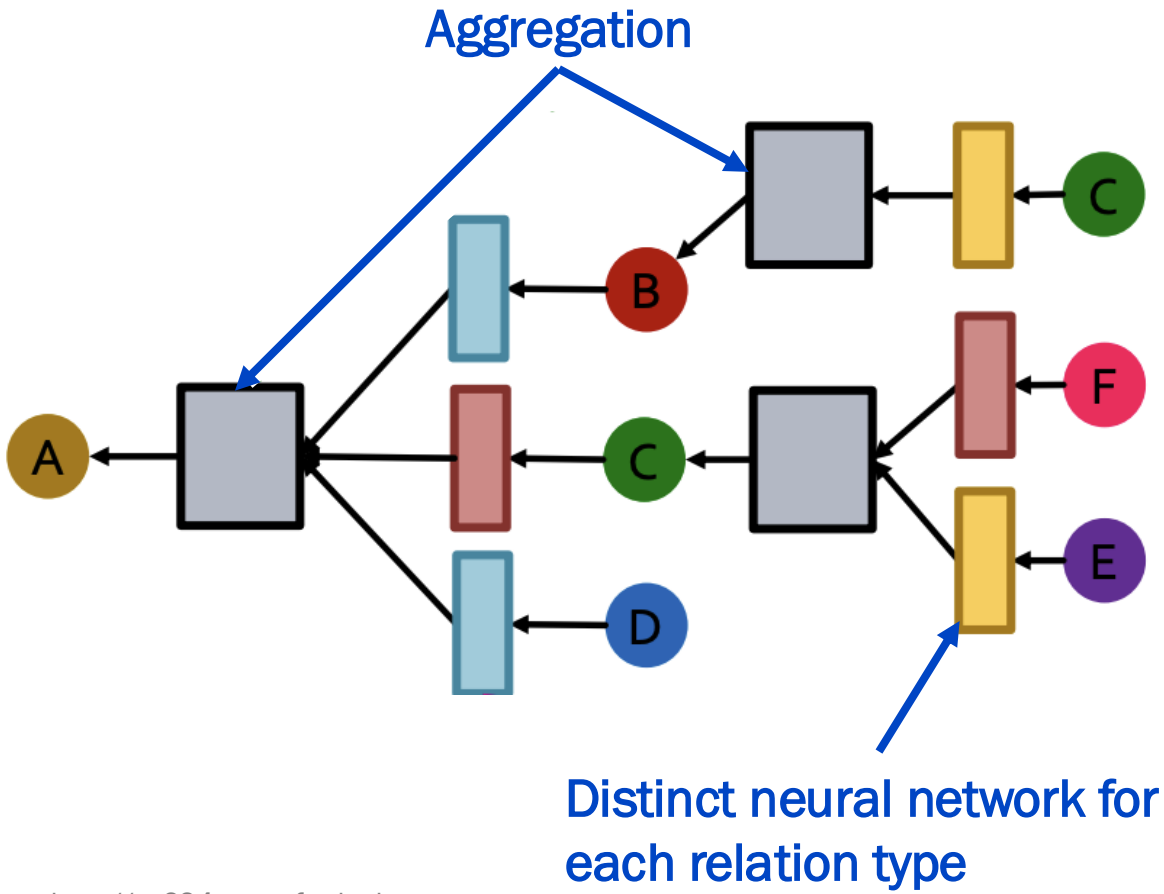


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational graph convolutional networks (RGCN)

- RGCN includes a neural network for each relation type
- In the node update, it sums the node embeddings for each relation

$$\mathbf{h}_v^{(k+1)} = \sigma\left(\left[\sum_{r\in R}\sum_{u\in N_r(v)}\frac{1}{c_{v,r}}\mathbf{W}_r^{(k)}\mathbf{h}_u^{(k)}\right] + \mathbf{W}_0^{(k)}\mathbf{h}_v^{(k)}\right)$$

where $N_r(v)$ is the neighborhood of node v for relation type r and $c_{v,r} = |N_r(v)|$

- In partial matrix form, the update for all nodes is given by

$$\mathbf{H}^{(k+1)} = \sigma\left(\left[\sum_{r\in R}\mathbf{D}_r^{-1}\mathbf{A}_r\mathbf{H}^{(k)}\mathbf{W}_r^{(k)}\right] + \mathbf{W}_0^{(k)}\mathbf{H}^{(k)}\right)$$

where $\mathbf{D}_r^{-1}$ is a diagonal matrix where the $(\mathbf{D}_r^{-1})_{v,v} = {}^{1}/_{|N_r(v)|}$

RGCN for edge prediction

- Assume *(E, r_3, A)* is a training supervision edge, and all other edges are training message edges
  - Other edges (not shown yet) are validation or test supervision edges
- Use the RGCN to predict probability for edge *(E, r_3, A)*
  - Take the final RGCN layer embeddings, $\mathbf{h}_E^{(K)}$ and $\mathbf{h}_A^{(K)}$, for *E* and *A*
  - Apply a relation specific prediction head g_r: $\mathbb{R}^{d_K \times d_K} \rightarrow \mathbb{R}$
    - Example: $g_{r_3} = \left(\mathbf{h}_E^{(K)}\right)^T \mathbf{W}_{r_3} \mathbf{h}_A^{(K)}$ where $\mathbf{W}_{r_3} \in \mathbb{R}^{d_K \times d_K}$

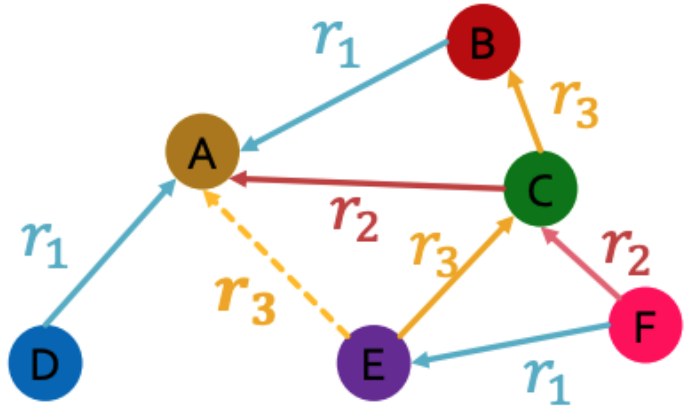


Image credit: Jure Leskovec, http://cs224w.stanford.edu

RGNNs for KGs with the same relation types

- RGCN can be used for inductive link prediction across new entities
  - Example
    - Training KG contains relation type: (User, Buys, Product)
    - Second KG also contains relation (User, Buys, Product) with different entities

RGNNs for KGs with the same relation types

- RGCN can be used for inductive link prediction across new entities
  - Example
    - Training KG contains relation type: (User, Buys, Product)
    - Second KG also contains relation (User, Buys, Product) with different entities
- RGCN cannot predict links for new relation types
  - Example
    - Training KG does not contain relation type: (User, Recommends, Product)
    - Second KG does contain relation type (User, Recommends, Product). The RGCN cannot predict links on this relation.

School of COMPUTING Clemson University

InGram

Truly foundational

- What if I want the model to transfer learning between KGs that have different entities and different relations?

Truly foundational

- What if I want the model to transfer learning between KGs that have different entities and different relations?

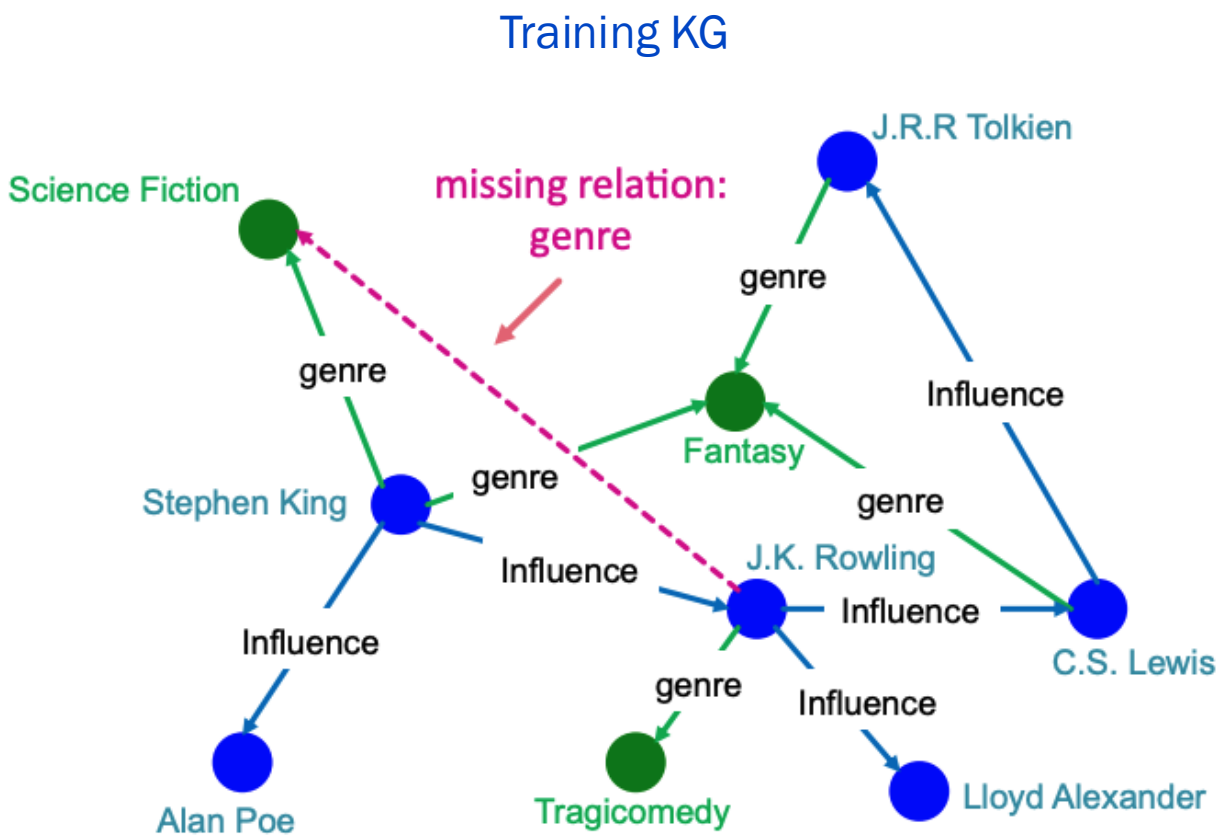


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Truly foundational

- What if I want the model to transfer learning between KGs that have different entities and different relations?

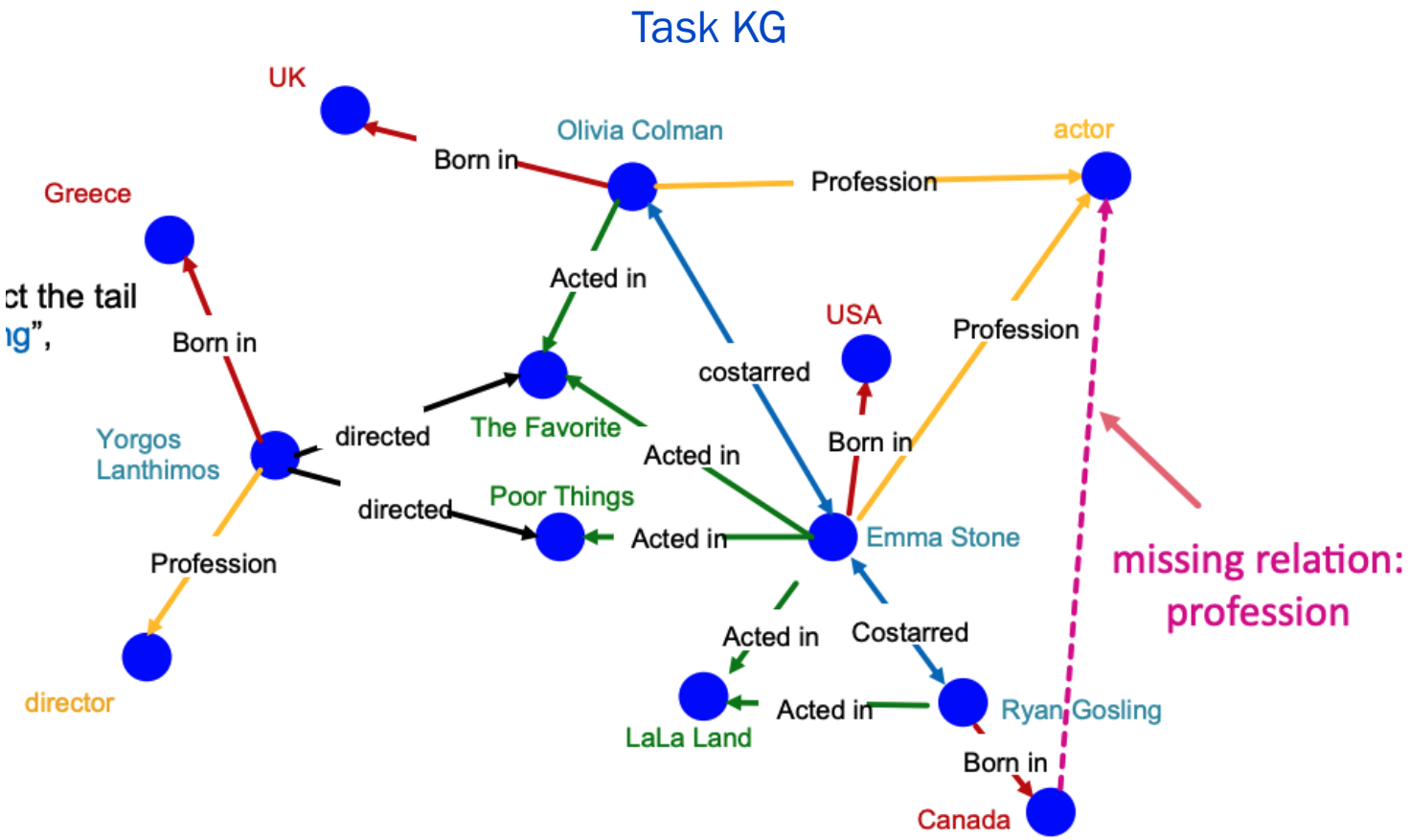


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Truly foundational

- What if I want the model to transfer learning between KGs that have different entities and different relations?
- We achieved entity equivariance by developing GNNs that learned how to form embeddings based on input graph structure
- We need to do something similar for relations
  - Utilize the *structure* of relations to let the model learn how to form relation embeddings from an input KG

InGram Model

Key idea is to create a ***relation graph*** representing the interaction of relations in the input graph

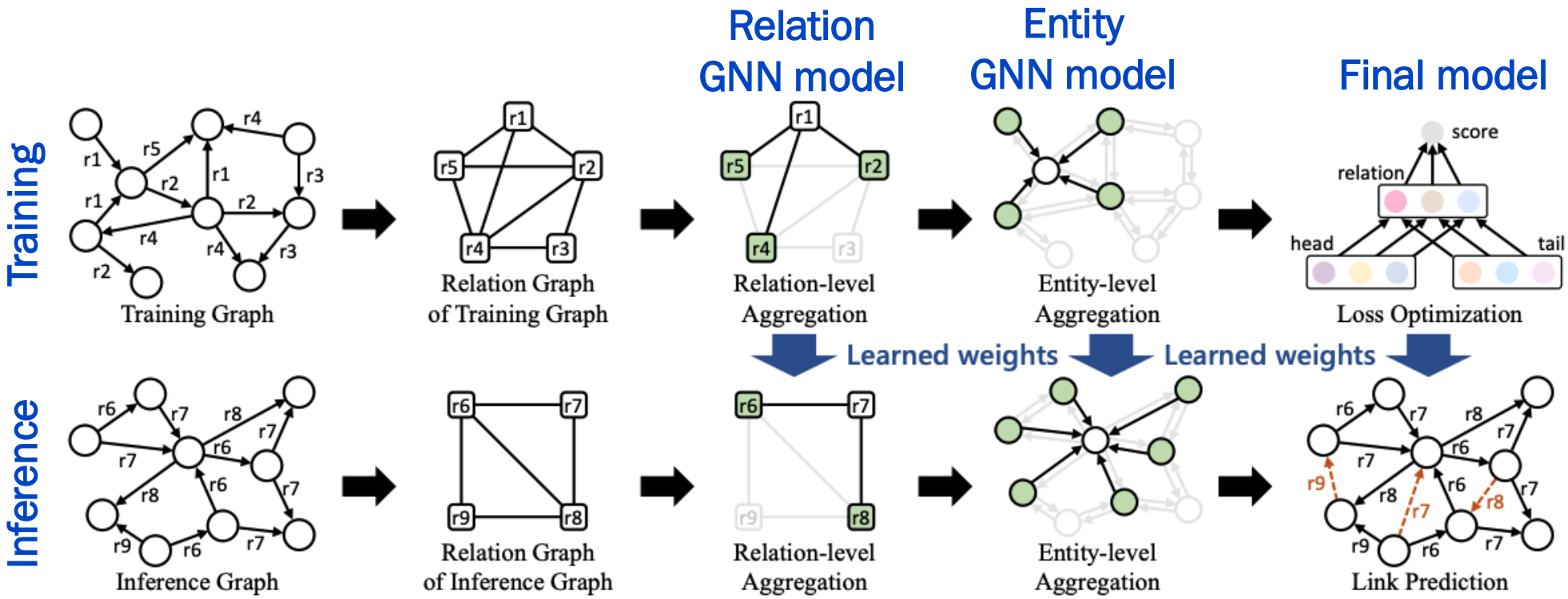


Image credit: Lee et al. 2023, https://arxiv.org/pdf/2305.19987

InGram Model

Key idea is to create a ***relation graph*** representing the interaction of relations in the input graph

1. How do we construct the relation graph?
2. How do we form the relation embeddings?
3. How do we form the entity embeddings?
4. What is the self-supervised task / loss function?

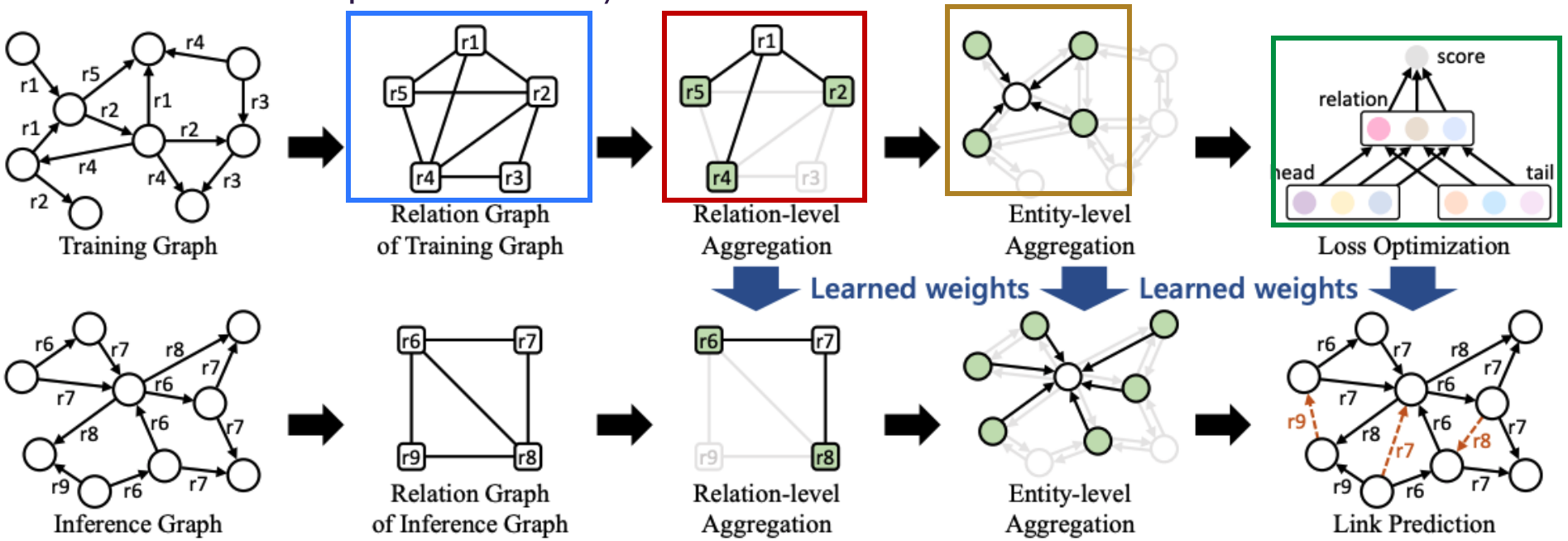
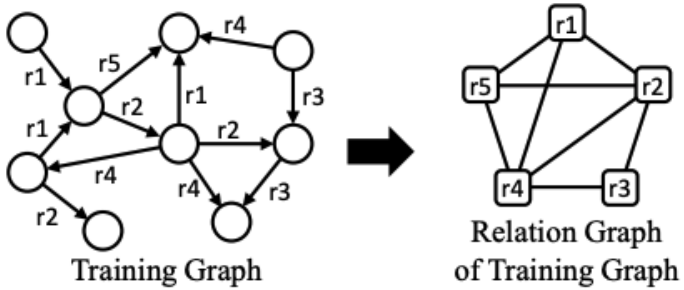


Image credit: Lee et al. 2023, https://arxiv.org/pdf/2305.19987

InGram relation graph



- Construct an affinity matrix
- Create two matrices

$$\boldsymbol{E}_h \in \mathbb{R}^{n \times m} \text{ and } \boldsymbol{E}_t \in \mathbb{R}^{n \times m}$$

where

- n is the number of entities (nodes) and m is the number of edge types
- $\boldsymbol{E}_h[i, j]$ is the number of times entity i is a head node for edge type j
- $\boldsymbol{E}_t[i, j]$ is the number of times entity i is a tail node for edge type j

https://github.com/masino-teaching/ml4g/blob/main/notebooks/ingram.ipynb

InGram relation graph

- Construct an affinity matrix

$$\boldsymbol{E}_h \in \mathbb{R}^{n \times m} \text{ and } \boldsymbol{E}_t \in \mathbb{R}^{n \times m}$$

  - $\boldsymbol{E}_h[i,j]$ is the number of times entity i is a head node for edge type j

```
[[0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 1. 1. 0. 1. 0.]
 [0. 0. 0. 1. 1. 1.]
 [0. 0. 0. 2. 1. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [2. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 1. 2. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]]
```

San Francisco
California
Star Wars
Hamill
Pitt
Eastwood
Catch Me
Director
USA
Spielberg
Private Ryan
Hanks
Actor

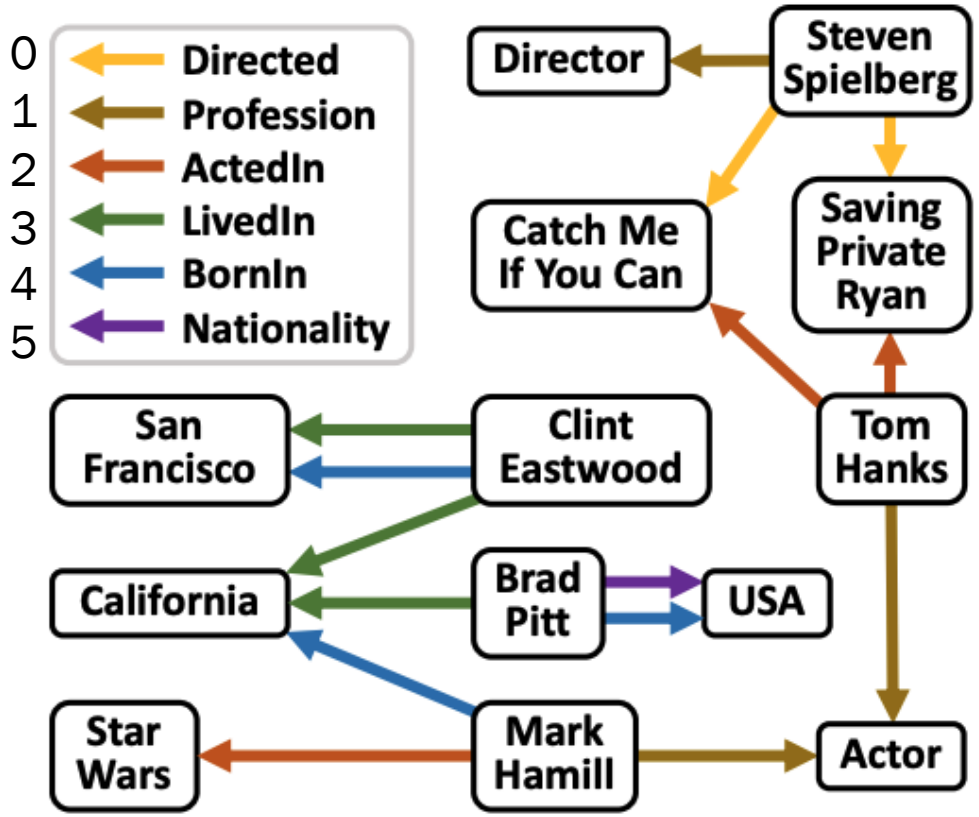


Image credit: Lee et al. 2023, https://arxiv.org/pdf/2305.19987

https://github.com/masino-teaching/ml4g/blob/main/notebooks/ingram.ipynb

InGram relation graph

- Construct an affinity matrix

$$\boldsymbol{E}_h \in \mathbb{R}^{n \times m} \text{ and } \boldsymbol{E}_t \in \mathbb{R}^{n \times m}$$

  - $\boldsymbol{E}_t[i,j]$ is the number of times entity i is a tail node for edge type j

```
[[0. 0. 0. 1. 1. 0.]
 [0. 0. 0. 2. 1. 0.]
 [0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [1. 0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 1.]
 [0. 0. 0. 0. 0. 0.]
 [1. 0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 2. 0. 0. 0. 0.]]
```

San Francisco
California
Star Wars
Hamill
Pitt
Eastwood
Catch Me
Director
USA
Spielberg
Private Ryan
Hanks
Actor

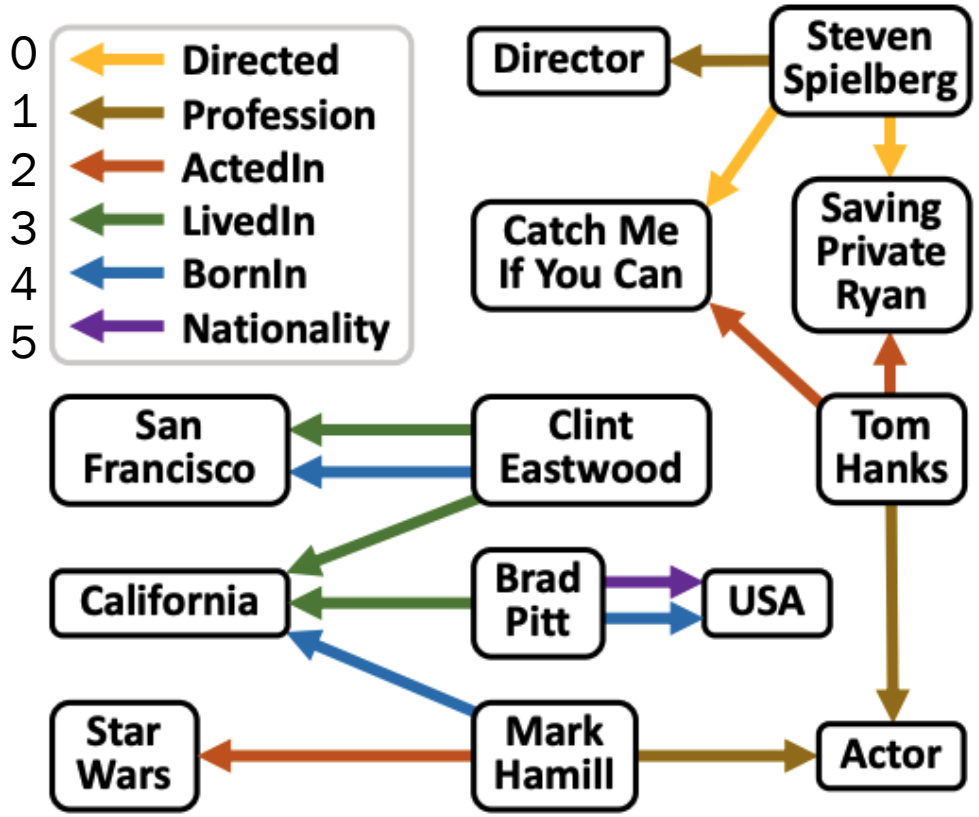
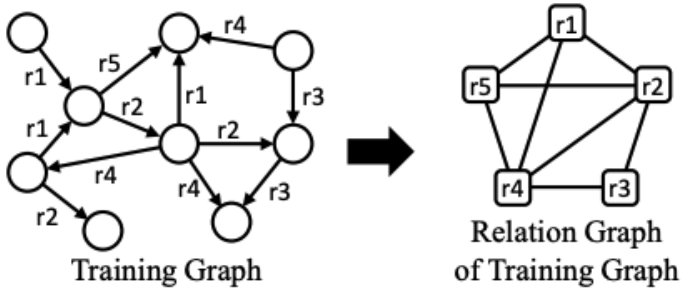


Image credit: Lee et al. 2023, https://arxiv.org/pdf/2305.19987

InGram relation graph



- Construct an affinity matrix
- Create two matrices

$$\boldsymbol{E}_h \in \mathbb{R}^{n \times m} \text{ and } \boldsymbol{E}_t \in \mathbb{R}^{n \times m}$$

- Define adjacency matrix, $\boldsymbol{A}$, for the relation types by

$$\boldsymbol{A}_h = \boldsymbol{E}_h^T \boldsymbol{D}_h^{-2} \boldsymbol{E}_h$$
$$\boldsymbol{A}_t = \boldsymbol{E}_t^T \boldsymbol{D}_t^{-2} \boldsymbol{E}_t$$
$$\boldsymbol{A} = \boldsymbol{A}_h + \boldsymbol{A}_h$$

where $\boldsymbol{D}_h \in \mathbb{R}^{n \times n}$ is the diagonal degree matrix such that $\boldsymbol{D}_h[i,i] = \sum_j \boldsymbol{E}_h[i,j]$ and $\boldsymbol{D}_t \in \mathbb{R}^{n \times n}$ is the diagonal degree matrix such that $\boldsymbol{D}_t[i,i] = \sum_j \boldsymbol{E}_t[i,j]$

- Note, $\boldsymbol{D}_h$ and $\boldsymbol{D}_t$ inverses can be computed via Moore-Penrose pseudo-inverse

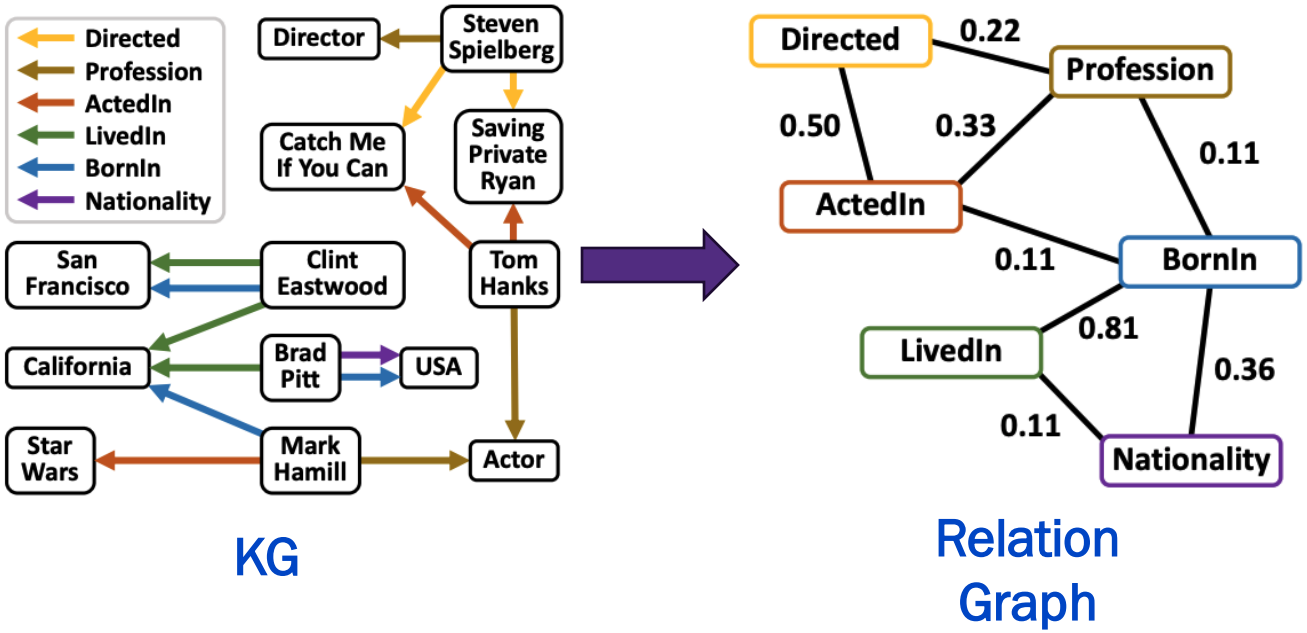
https://github.com/masino-teaching/ml4g/blob/main/notebooks/ingram.ipynb

https://github.com/masino-teaching/ml4g/blob/main/notebooks/ingram.ipynb

InGram relation graph

- Weighted adjacency matrix defines relation graph

$$A = E_h^T D_h^{-2} E_h + E_t^T D_t^{-2} E_t$$




```
[[0.94 0.22 0.5  0.   0.   0.  ]
 [0.22 2.33 0.33 0.   0.11 0.  ]
 [0.5  0.33 2.06 0.   0.11 0.  ]
 [0.   0.   0.   1.25 0.81 0.11]
 [0.   0.11 0.11 0.81 0.94 0.36]
 [0.   0.   0.   0.11 0.36 0.36]]
```

Relation adjacency matrix

Image credit: Lee et al. 2023, https://arxiv.org/pdf/2305.19987

InGram Model

Key idea is to create a ***relation graph*** representing the interaction of relations in the input graph

1. ✔ How do we construct the relation graph?
2. How do we form the relation embeddings?
3. How do we form the entity embeddings?
4. What is the self-supervised task / loss function?

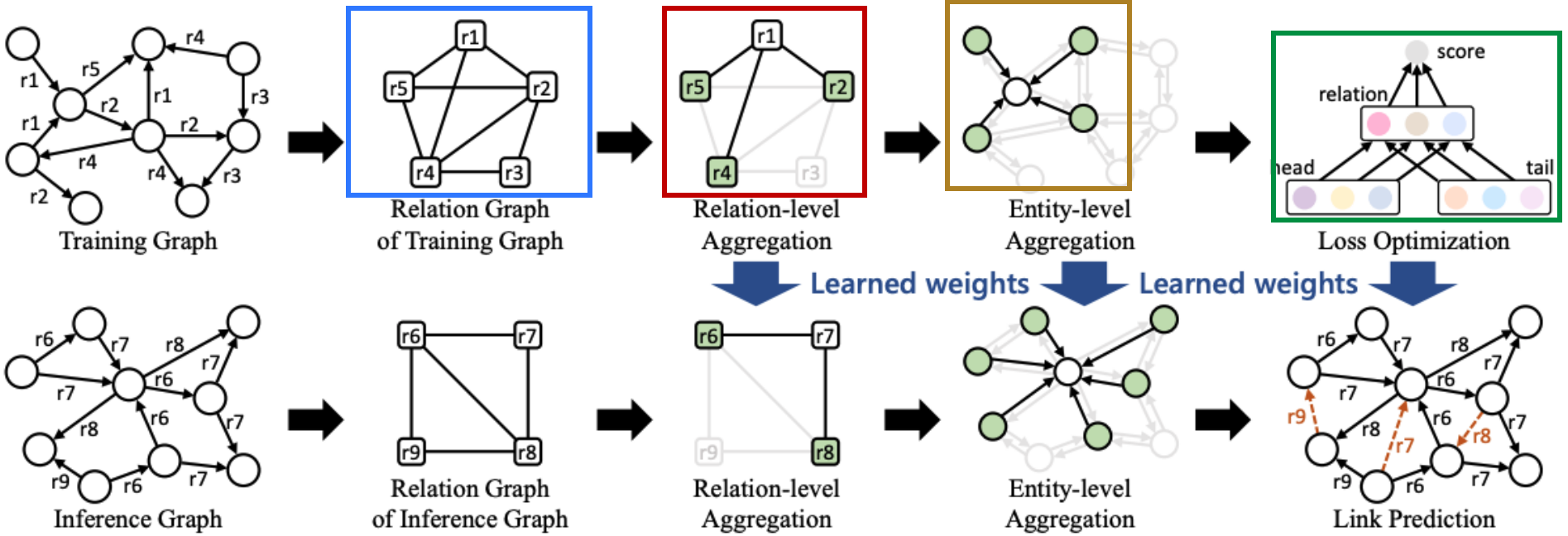
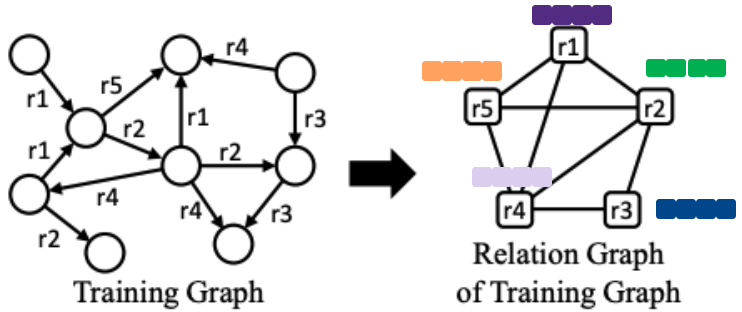


Image credit: Lee et al. 2023, https://arxiv.org/pdf/2305.19987

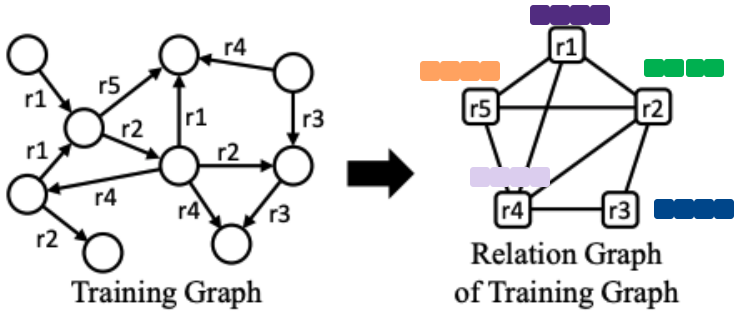
InGram relation embeddings



- Given the weighted relation adjacency matrix $\boldsymbol{A}$ we do the following:
  1. Randomly initialize relation embeddings, $\boldsymbol{x}_i \in \mathbb{R}^d$, for each relation r_i
  2. Set $\boldsymbol{z}_i^{(0)} = \boldsymbol{W}\boldsymbol{x}_i$ where $\boldsymbol{W} \in \mathbb{R}^{d' \times d}$ is a learnable matrix
  3. Construct a graph attention network with L layers where the relation embedding update at each layer is given by

$$\boldsymbol{z}_i^{(l+1)} = \sigma\left(\sum_{r_j \in \mathcal{N}_i} \alpha_{ij}^{(l)} \boldsymbol{W}^l \boldsymbol{z}_j^{(l)}\right)$$

InGram relation embeddings



1. Construct a graph attention network with L layers where the relation embedding update at each layer is given by

$$\mathbf{z}_i^{(l+1)} = \sigma\left(\sum_{r_j \in \mathcal{N}_i} \alpha_{ij}^{(l)} \boldsymbol{W}^l \mathbf{z}_j^{(l)}\right)$$

Learnable parameters

$$\alpha_{ij}^{(l)} = \frac{\exp\left(\mathbf{y}^{(l)} \sigma\left(\boldsymbol{P}^{(l)}\left[\mathbf{z}_i^{(l)} \| \mathbf{z}_j^{(l)}\right]\right) + c_{s(i,j)}^{(l)}\right)}{\sum_{r_{j'} \in \mathcal{N}_i} \exp\left(\mathbf{y}^{(l)} \sigma\left(\boldsymbol{P}^{(l)}\left[\mathbf{z}_i^{(l)} \| \mathbf{z}_{j'}^{(l)}\right]\right) + c_{s(i,j')}^{(l)}\right)}$$

Index on $c^{(l)}$ which is a measure of affinity between (i, j) considered globally (i.e, against all other relation pairs

$$s(i,j) = \left\lceil \frac{\text{rank}(a_{ij}) \times B}{\text{nnz}(\boldsymbol{A})} \right\rceil$$

$a_{ij} \in \boldsymbol{A}$ (relation adjacency matrix), $rank(a_{ij})$ is the ranking of a_{ij} among the non-zero elements of $\boldsymbol{A}$ sorted in descending order, $nnz(\boldsymbol{A})$ is the number of non-zero elements in $\boldsymbol{A}$, and B is the number of bins (hyperparameter)

InGram Model

Key idea is to create a ***relation graph*** representing the interaction of relations in the input graph

1. ✔ How do we construct the relation graph?
2. ✔ How do we form the relation embeddings?
3. How do we form the entity embeddings?
4. What is the self-supervised task / loss function?

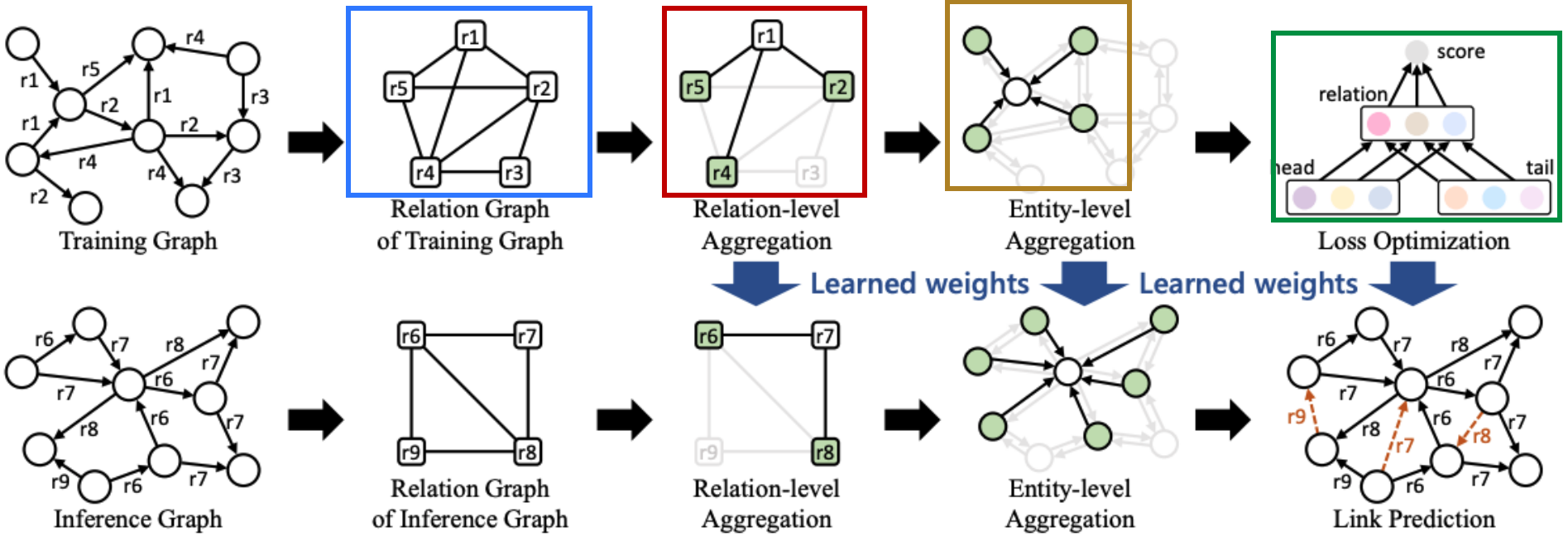
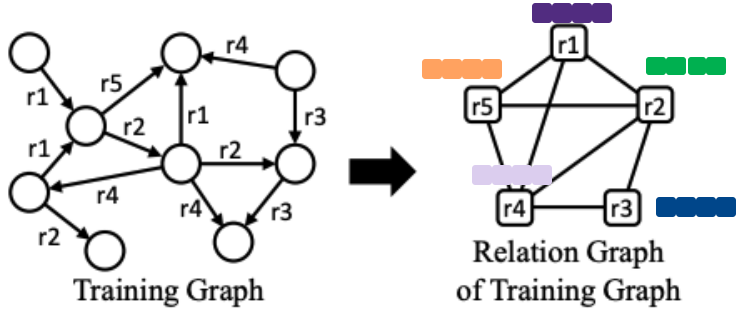


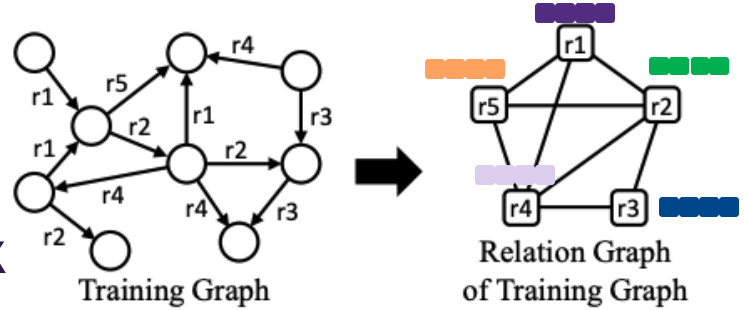
Image credit: Lee et al. 2023, https://arxiv.org/pdf/2305.19987

InGram entity embeddings



- Assume we have the final relation embeddings, $\boldsymbol{z}_i^{(L)}$
- Consider an entity (node in the KG), v_i
- Define the neighborhood, $\widehat{\mathcal{N}}_i$ to be all nodes v_j such that the triple (v_j, r_k, v_i) is in the KG (i.e., v_i has an inward edge from v_i for some relation r_k
- Randomly initialize node embeddings, $\hat{\boldsymbol{x}}_{\boldsymbol{i}} \in \mathbb{R}^{\hat{d}}$, for each entity v_i
- Set $\boldsymbol{h}_i^{(0)} = \widehat{\boldsymbol{W}}\hat{\boldsymbol{x}}_{\boldsymbol{i}}$ where $\widehat{\boldsymbol{W}} \in \mathbb{R}^{\hat{d}' \times \hat{d}}$ is a learnable matrix

InGram entity embeddings



- Set $\boldsymbol{h}_i^{(0)} = \widehat{\boldsymbol{W}}\hat{\boldsymbol{x}}_i$ where $\widehat{\boldsymbol{W}} \in \mathbb{R}^{\hat{d}' \times \hat{d}}$ is a learnable matrix
- Construct a graph attention network with L layers where the entity embedding update at each layer is given by

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\beta_{ii}^{(l)}\widehat{\boldsymbol{W}}^{(l)}[\mathbf{h}_i^{(l)} \| \bar{\mathbf{z}}_i^{(L)}] + \sum_{v_j \in \widehat{\mathcal{N}}_i}\sum_{r_k \in \mathcal{R}_{ji}} \beta_{ijk}^{(l)}\widehat{\boldsymbol{W}}^{(l)}[\mathbf{h}_j^{(l)} \| \mathbf{z}_k^{(L)}]\right)$$

Learnable matrix

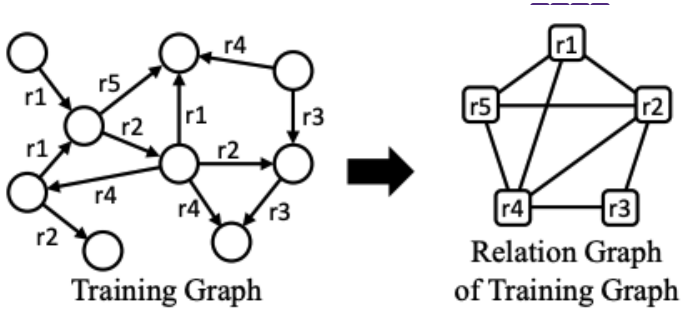
Self loop

Neighbor aggregation with attention

$$\bar{\mathbf{z}}_i^{(L)} = \sum_{v_j \in \widehat{\mathcal{N}}_i}\sum_{r_k \in \mathcal{R}_{ji}} \frac{\mathbf{z}_k^{(L)}}{\sum_{v_{j'} \in \widehat{\mathcal{N}}_i} |\mathcal{R}_{j'i}|}$$

Mean of the **final** relation embeddings for the relations incident to v_i

InGram entity embeddings



Neighbor aggregation with attention

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\beta_{ii}^{(l)}\widehat{\boldsymbol{W}}^{(l)}[\mathbf{h}_i^{(l)}\|\bar{\mathbf{z}}_i^{(L)}] + \sum_{v_j\in\widehat{\mathcal{N}}_i}\sum_{r_k\in\mathcal{R}_{ji}}\beta_{ijk}^{(l)}\widehat{\boldsymbol{W}}^{(l)}[\mathbf{h}_j^{(l)}\|\mathbf{z}_k^{(L)}]\right)$$

Learnable parameters

$$\beta_{ii}^{(l)} = \frac{\exp\left(\hat{\mathbf{y}}^{(l)}\sigma\left(\widehat{\boldsymbol{P}}^{(l)}\mathbf{b}_{ii}^{(l)}\right)\right)}{\exp\left(\hat{\mathbf{y}}^{(l)}\sigma\left(\widehat{\boldsymbol{P}}^{(l)}\mathbf{b}_{ii}^{(l)}\right)\right) + \sum_{v_{j'}\in\widehat{\mathcal{N}}_i}\sum_{r_{k'}\in\mathcal{R}_{j'i}}\exp\left(\hat{\mathbf{y}}^{(l)}\sigma\left(\widehat{\boldsymbol{P}}^{(l)}\mathbf{b}_{ij'k'}^{(l)}\right)\right)}$$

$$\mathbf{b}_{ii}^{(l)} = [\mathbf{h}_i^{(l)}\|\mathbf{h}_i^{(l)}\|\bar{\mathbf{z}}_i^{(L)}]$$

$$\beta_{ijk}^{(l)} = \frac{\exp\left(\hat{\mathbf{y}}^{(l)}\sigma\left(\widehat{\boldsymbol{P}}^{(l)}\mathbf{b}_{ijk}^{(l)}\right)\right)}{\exp\left(\hat{\mathbf{y}}^{(l)}\sigma\left(\widehat{\boldsymbol{P}}^{(l)}\mathbf{b}_{ii}^{(l)}\right)\right) + \sum_{v_{j'}\in\widehat{\mathcal{N}}_i}\sum_{r_{k'}\in\mathcal{R}_{j'i}}\exp\left(\hat{\mathbf{y}}^{(l)}\sigma\left(\widehat{\boldsymbol{P}}^{(l)}\mathbf{b}_{ij'k'}^{(l)}\right)\right)}$$

$$\mathbf{b}_{ijk}^{(l)} = [\mathbf{h}_i^{(l)}\|\mathbf{h}_j^{(l)}\|\mathbf{z}_k^{(L)}]$$

Final relation embeddings from relation graph GNN

InGram Model

Key idea is to create a ***relation graph*** representing the interaction of relations in the input graph

1. ✔ How do we construct the relation graph?
2. ✔ How do we form the relation embeddings?
3. ✔ How do we form the entity embeddings?
4. What is the self-supervised task / loss function?

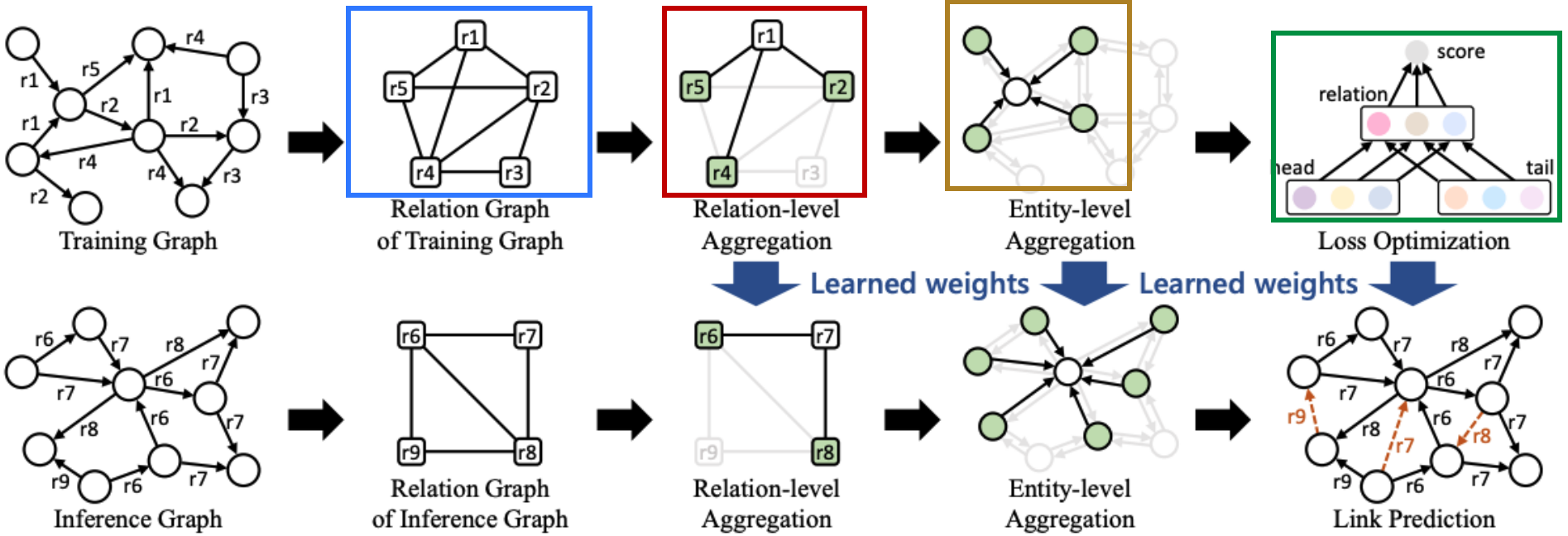


Image credit: Lee et al. 2023, https://arxiv.org/pdf/2305.19987

InGram prediction head

Learnable parameters

- Final model outputs
  - Relation r_k embedding $\boldsymbol{z}_k = \boldsymbol{M}\boldsymbol{z}_k^{(L)}$
  - Entity v_i embedding $\boldsymbol{h}_i = \widehat{\boldsymbol{M}}\boldsymbol{h}_i^{(\hat{L})}$
- Scoring function for tail completion

$$f(v_i, r_k, v_j) = \boldsymbol{h}_i^T \, diag(\overline{\boldsymbol{W}}\boldsymbol{z}_k)\boldsymbol{h}_j$$

- Contrastive margin loss

$$\sum_{(v_i, r_k, v_j) \in \mathcal{T}_{\text{tr}}} \sum_{(\mathring{v}_i, r_k, \mathring{v}_j) \in \mathring{\mathcal{T}}_{\text{tr}}} \max(0, \gamma - f(v_i, r_k, v_j) + f(\mathring{v}_i, r_k, \mathring{v}_j))$$

InGram Model

Key idea is to create a ***relation graph*** representing the interaction of relations in the input graph

1. ✔ How do we construct the relation graph?
2. ✔ How do we form the relation embeddings?
3. ✔ How do we form the entity embeddings?
4. ✔ What is the self-supervised task / loss function?

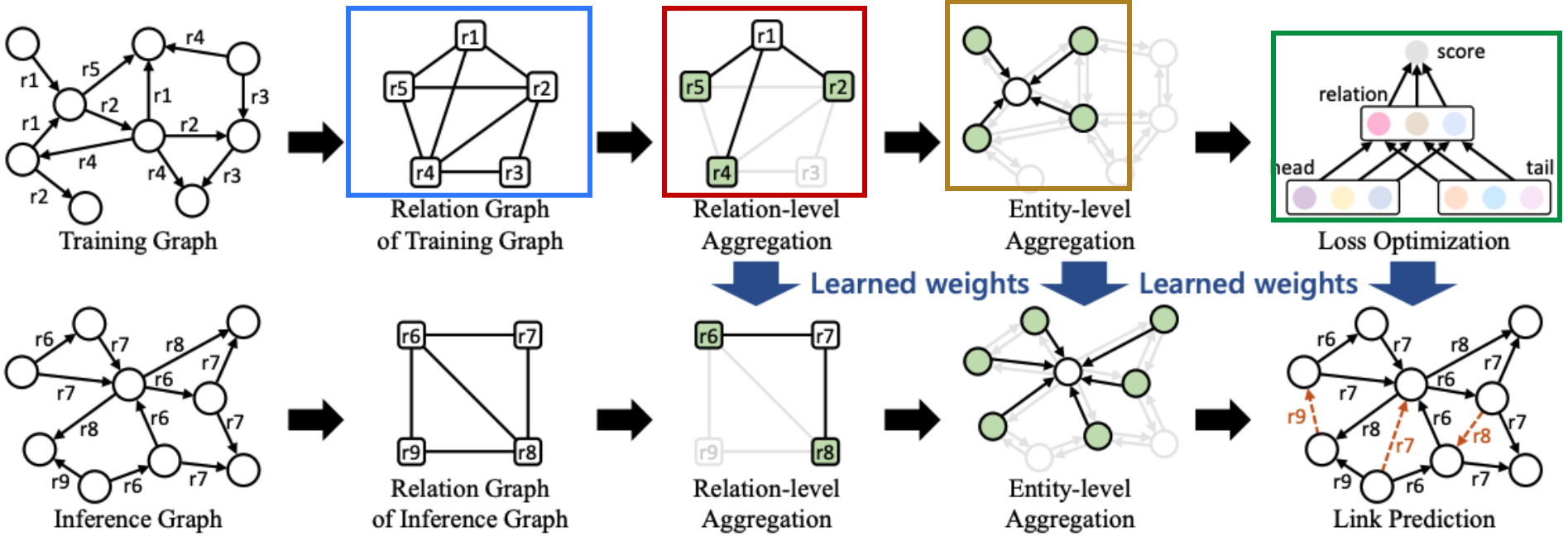


Image credit: Lee et al. 2023, https://arxiv.org/pdf/2305.19987

InGram

So what did this get us?

- GNN learns how to form entity embeddings from graph structure → inductive on entities
- GNN learns how to form relation embeddings from graph structure → inductive on relations
- Can be used for inductive link prediction on new entities and new relations
  - It is a method that can yield foundation models
- Limitations
  - Use of degree statistics for relation graph weights causes InGram to be *distributionally double equivariant* – i.e., over many permutations the average relation embeddings are double equivariant, but any given permutation may not be
  - See Zhou 2025 for more - https://arxiv.org/pdf/2302.01313